

Hybrid Static–Trace Analysis for Interaction Model Synthesis in Event-Driven Systems

Scenario-Specific Interaction Diagrams in SysML v2

Prathik Anand Krishnan

Hybrid Static–Trace Analysis for Interaction Model Synthesis in Event-Driven Systems

THESIS

submitted in partial fulfilment of the
requirements for the degree of

MASTER OF SCIENCE

in

SOFTWARE ENGINEERING

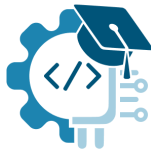
by

Prathik Anand Krishnan

under the supervision of

Dr. L. Thomas van Binsbergen (CCI, UvA)

July 2026



Master Software Engineering
Informatics Institute
FNWI, University of Amsterdam
Amsterdam, The Netherlands



TNO-ESI
High Tech Campus 25
5656 AE Eindhoven
The Netherlands

Thesis Committee

Examiner (chair): Dr. L. Thomas van Binsbergen (CCI, UvA)
Second Reviewer: Dr. Prof. Benny Åkesson (PCS, UvA)
Daily Supervisor: Dr. Rosilde Corvino (TNO-ESI)

Abstract

Software systems in safety-critical domains such as medical imaging and industrial automation evolve continuously, making manually maintained behavioural documentation increasingly difficult to keep up to date. SysML v2 Interaction Models, structure and sequence diagrams that capture the components involved in a scenario and the interactions between them, are a natural target for automated reconstruction, but existing reconstruction approaches rely on partial information sources and therefore provide an incomplete view of system interactions: static analysis over-approximates possible behaviour and cannot by itself determine which interactions are exercised during a specific execution, while execution traces provide execution-specific information but only for behaviour exercised during the observed runs. This thesis investigates: (RQ₁) whether combining a static call graph with runtime execution traces reconstructs scenario-specific Interaction Models more accurately than either source alone; (RQ₂) whether guiding trace collection with the static call graph removes reconstruction noise without losing scenario coverage; and (RQ₃) how well a reconstructed SysML v2 Interaction Model consolidates static and dynamic evidence into a single artefact sufficient for system-behaviour comprehension.

Methods: A hybrid pipeline was built that synthesises Interaction Models for event-driven C++ systems by combining a static call graph with timestamped runtime traces collected by a separate pipeline. Static and dynamic evidence were maintained as separate artefacts due to their different evolution rates and storage characteristics; the call graph, stored in a Neo4j property graph, contains only trace anchors, preserving traceability without embedding large execution traces. The scenario-scoped Cypher queries isolated the subset of static interactions associated with a given scenario, where relevance was established through trace analysis and corresponding static call-graph anchors. The pipeline was implemented using tools that, if required, can be invoked via twelve composable, natural-language-invokable agent skills. The tool was evaluated on *CPSCore*, an open-source event-driven framework, across four found-in-the-wild scenarios drawn from its existing test suite. Each scenario was reconstructed under nine conditions (C₁–C₉), five of them fully deterministic and reproducible (C₂–C₆): an Large Language Model (LLM)-only baseline (C₁); static call-graph traversal (C₂); unfiltered dynamic trace (C₃); guided instrumentation only (C₄); guided instrumentation with a runtime reachability filter (C₅); full-trace instrumentation with the same filter (C₆); and

three LLM-augmented conditions (C7–C9) that post-process C2, C3, and C4 candidates respectively. A blinded six-rater expert panel compared diagrams reconstructed from unfiltered trace output against diagrams reconstructed from trace output scoped using static call-graph analysis of the scenario. The underlying static approach was further tested in a practitioner session on a second, larger industrial codebase to assess industrial usefulness beyond reconstruction accuracy.

Results: Guided instrumentation with a Neo4j runtime reachability filter (C5) achieved perfect F1 on both interaction-set and execution-order metrics across all four scenarios (macro-F1 = 1.000 / 1.000). Full-trace instrumentation with the same filter (C6) matched C5 exactly, showing the filter is sufficient regardless of instrumentation strategy. Guided instrumentation without the filter (C4) is the strongest Neo4j-free deterministic baseline (macro-F1 = 0.914), while unfiltered dynamic tracing (C3) was the weakest (0.530), dominated by intra-component false positives. The LLM-only baseline (C1) reached macro-F1 = 0.964, failing only on S3 where a teardown-phase call appears in the source snippet. LLM augmentation of evidence priors (C7–C9) showed diminishing returns as the prior improved: C7 substantially recovered C2’s recall on three of four scenarios; C8 partially denoised C3; C9 degraded C4’s already-precise output (0.783 vs. C4’s 0.914), confirming that LLM post-processing is counterproductive when the prior is already clean. Code-graph-guided instrumentation eliminated all noise (50% fewer nodes, 70% fewer edges) without losing scenario coverage, and was rated higher by the expert panel in 47 of 54 comparisons. In a scenario-comprehension task, no single evidence source alone, including a reconstructed diagram, was sufficient: source code alone was weakest, missing components and interactions that trace or diagram evidence made explicit, while a diagram alone tied with runtime traces alone; only combining source code with runtime traces reached perfect recall. In the practitioner session, structural and dependency diagrams were rated consistently useful, while sequence-diagram message labelling was identified as the clearest area for improvement.

Evaluation and Conclusion: The three results converge on one pattern: additional evidence improves reconstruction or comprehension specifically when the primary evidence source is missing something, and is otherwise redundant. The thesis contributes an empirical assessment of hybrid static and dynamic analysis for reconstructing scenario-specific SysML v2 diagrams. The results show when combining static and dynamic evidence improves reconstruction, when scenario-guided instrumentation reduces reconstruction noise without sacrificing coverage, and to what extent reconstructed diagrams can replace the underlying evidence in program-comprehension tasks. The work further provides an implementation comprising a scenario-scoping method, a reversible instrumentation tool (`csp_matcher`), and an automated workflow based on natural-language agent skills.

Contents

Abstract	iv
Contents	vi
List of Figures	x
List of Tables	xii
Acknowledgements	xiii
1 Introduction	1
1.1 Context	1
1.2 Problem Statement	2
1.3 Research Questions	2
1.4 Approach	3
1.5 Outline	3
2 Background	4
2.1 SysML v2 and Interaction Models	4
2.1.1 Sequence Diagrams in SysML v2	4
2.2 Property Graphs	5
2.3 Static Call-Graph Analysis	5
2.3.1 Clang and the Renaissance Semantic Extractor	6
2.3.2 Limitations in Event-Driven Systems	6
2.4 Execution Trace Analysis	6
2.4.1 Challenges with Incomplete Traces	6
2.5 Scenario Scoping through Graph Queries	7
2.6 Publish-Subscribe via Callback Registration	7
3 Related Work	8
3.1 Trace-Based Interaction Model Extraction	8
3.1.1 Limitations	9

3.2	Scenario-Driven and Functionality-Centric Analysis	9
3.3	Behavioural Pattern Mining and Model Inference	9
3.4	Static Analysis for Architecture Recovery	9
3.4.1	Handling Event-Driven Dispatch	10
3.5	Hybrid Static and Dynamic Analysis for Behaviour Recovery	10
3.6	Trace Visualisation for Program Comprehension	11
3.7	Summary and Positioning	11
4	Methods	12
4.1	Pipeline Overview	12
4.2	Property Graph Schema	14
4.2.1	Design Decision	14
4.2.2	Node Types	14
4.2.3	Relationship Types	14
4.3	Agent Skill Architecture	15
4.4	Pattern-Based Instrumentation (csp_matcher)	16
4.4.1	Design	17
4.4.2	Integration with the Pipeline	17
4.4.3	Reversibility via Filter Functions	18
4.5	Macro Resolution for Publish-Subscribe Systems	18
4.6	Scenario Scoping	19
4.6.1	Component Constraint Filter	19
4.6.2	Neo4j-Based Reachability Filter	20
4.7	Ground-Truth Derivation	20
4.8	SysML v2 Generation	23
5	Implementation	24
5.1	Representative System: CPSCore	24
5.1.1	Scale and Dispatch Characteristics	24
5.1.2	CPSCore Architecture	25
5.1.3	Publish-Subscribe via IDC and Redis	25
5.2	Phase 1 — Static Analysis	25
5.2.1	Renaissance to Neo4j	25
5.2.2	Schema Introspection Skill	25
5.2.3	Scenario-Scoped Query	26
5.2.4	Dependency Table Extraction	26
5.3	Phase 2 — Guided Instrumentation	26
5.3.1	Probe Insertion via csp_matcher	26
5.3.2	Trace Format and Output	27
5.4	Phase 3 — Neo4j-Based Reachability Filter	27
5.5	Phase 4 — SysML v2 Generation	27
5.6	Scenario Suite and Experimental Conditions	30
5.6.1	Scenario Suite (S1a, S1b, S2, S3)	30
5.6.2	Nine-Condition Design (C1–C9)	31
5.6.3	Guided versus Full Instrumentation	32

6	Evaluation	33
6.1	Evaluation Design	33
6.2	H1: Does Combining Static and Dynamic Evidence Improve Reconstruc- tion Accuracy?	33
6.2.1	Metric	34
6.2.2	Results	34
6.2.3	Discussion: C5 and C6 Achieve Perfect Scores	35
6.2.4	Discussion: Guided vs. Unfiltered and Static Limits	36
6.2.5	Discussion: LLM-Augmented Conditions Underperform Their Counterparts	36
6.3	H2: Does Guided Instrumentation Improve Scenario Focus?	36
6.3.1	Metrics	37
6.3.2	Quantitative Results	37
6.3.3	Qualitative Arm: Blinded Expert Rating	37
6.4	H3: Do Reconstructed Models Improve Task Performance?	38
6.4.1	Protocol	38
6.4.2	Holistic Comparison	39
6.4.3	H3 Conclusion	39
6.5	Practitioner Validation: A Second Industrial Codebase	40
6.5.1	Session Design	40
6.5.2	Scenario Exercise: A Real Engineering Issue	40
6.5.3	Interview Findings	41
6.5.4	Relationship to This Thesis's Contribution	42
6.6	Discussion: Cross-Cutting Pattern	42
6.7	Answering the Research Questions	43
6.8	Engineering Utility	44
6.9	Threats to Validity	45
6.9.1	Internal Validity	45
6.9.2	External Validity	45
6.9.3	Construct Validity	45
7	Conclusion	46
7.1	Summary & Findings	46
7.2	Practical Implications	48
7.3	Contributions	48
7.4	Limitations & Threats to Validity	48
7.5	Future Work	49
7.6	Reproducibility	50
A	Appendix	51
A.1	Property Graph Schema: Full Cypher Constraints	51
A.2	csp_matcher Tracing Rules for CPSCore	51
A.3	Ground-Truth Reference Diagrams (S1, S2, S3, S4)	52
A.3.1	S1: Aggregator Notification Chain, Structural	53
A.3.2	S1: Aggregator Notification Chain, Sequence	54
A.3.3	S2: Configuration Property Mapping, Structural	54

A.3.4	S2: Configuration Property Mapping, Sequence	55
A.3.5	S3: Multi-Component Runner Orchestration, Structural	55
A.3.6	S3: Multi-Component Runner Orchestration, Sequence	57
A.3.7	S4: Aggregator Notification Chain, Timeout Path, Structural .	57
A.3.8	S4: Aggregator Notification Chain, Timeout Path, Sequence . .	59
A.4	Generated Pipeline Output	59
A.4.1	C5 (Intersection) Edge Lists by Scenario	59
A.4.2	Generated SysML v2 Sequence Model (Excerpt)	60
A.5	Generated Mermaid Diagrams	60
A.6	Rendered Diagram Figures	61
A.6.1	Rendered Diagrams: S1, S2, S3, S4 (Illustrative)	61
A.7	H2 Qualitative Arm: Per-Rater Decomposition	67
A.8	SysML v2 Generated Diagrams	68
A.8.1	Structural Diagrams	68
A.8.2	Sequence Diagrams	70
References		73
Acronyms		76

List of Figures

4.1	The concrete C5 pipeline as run for the H1 experiments, with the actual tools and artefacts behind each stage: Renaissance populating the Neo4j code graph and computing a scenario-scoped reachable function set via CppCalls*1..8 traversal (Phase 1); guided instrumentation producing <code>trace_slice.txt</code> at architecture-level call sites (Phase 2); the Neo4j-based reachability filter scoping <code>trace_slice.txt</code> to callers in the reachable function set and writing <code>interactions.json</code> and <code>sequence.json</code> (Phase 3, condition C5); and <code>gen_sysml.py</code> rendering the final SysML v2 v2 output (Phase 4, sections 4.6.2 and 4.8).	13
4.2	Ground-truth derivation pipeline. An architect manually inspects the C++ source and draws Excalidraw structural and sequence diagrams for each scenario (section A.3). These diagrams serve two roles: they identify the architecture-level call sites passed to Renaissance and <code>clang-exp</code> for guided instrumentation, and they define the <code>SCENARIO_ENTRY</code> entry-point and <code>REACHABILITY</code> component-pair filters encoded in <code>arch_filters.py</code> . The instrumented test run produces <code>trace_slice.txt</code> ; <code>derive_gt_arch.py</code> then applies both filters and deduplicates to <code>gt_interactions.json</code> and <code>gt_sequence.json</code>	22
5.1	An example of the pipeline’s final rendered output: the generated SysML v2 sequence diagram for the whole-codebase <i>Aggregation</i> scenario, one of the 14 bulk-generated, unscoped diagrams described above. It is illustrative of output style and coverage only, not one of the scenario-scoped, per-condition reconstructions scored in section 6.2 (those are reproduced separately in section A.4). The complete set of bulk-generated diagrams is reproduced in section A.8.	29
A.1	Source annotation for S1: the Synchronization/Aggregation/Logging call chain exercised by <code>SynchronizedRunnerTest</code> ’s happy path.	53
A.2	S2 ground truth, drawn from direct source inspection: the Configuration-to-Logging linear property-mapping chain.	55

A.3	S ₃ ground truth, drawn from direct source inspection: the multi-component fan-out from <code>AggregatableRunner</code> into Aggregation, Utilities and Logging (section 5.6.1).	56
A.4	Source annotation for S ₄ : the same call chain as S ₁ , exercised instead by <code>SynchronizedRunnerTest</code> 's timeout branch, which additionally logs via <code>RAIILogStream.stream()</code>	58
A.5	Module-level dependency structure of CPSCore (cross-directory call edges).	62
A.6	Cross-module function-call dependencies to singleton APIs.	62
A.7	SignalHandler subsystem: static call edges (solid) and runtime subscriptions (dashed).	63
A.8	S ₁ (Aggregator notification chain): interconnection diagram.	63
A.9	S ₁ sequence diagram: message interactions.	64
A.10	S ₂ (Configuration property mapping): interconnection diagram.	64
A.11	S ₂ sequence diagram: Configuration property mapping.	65
A.12	S ₃ (multi-component runner orchestration): interconnection diagram.	65
A.13	S ₃ sequence diagram: multi-component runner orchestration.	66
A.14	S ₄ (Aggregator notification chain, timeout path): interconnection diagram.	66
A.15	S ₄ sequence diagram: Aggregator notification chain, timeout path.	67
A.16	Complete CPSCore component model: all part definitions and interface connections.	68
A.17	Node-type hierarchy and property graph schema relationships (13 node labels).	69
A.18	SysML v2 sequence diagrams: Aggregation (left) and Synchronization (right).	69
A.19	SysML v2 sequence diagrams: IDC (left) and IPC (right).	70
A.21	SysML v2 sequence diagrams: Signal handler (left) and Test (right).	70
A.20	SysML v2 sequence diagrams: Redis (left) and Serial (right).	71
A.22	SysML v2 sequence diagrams: Utilities (left) and Data presentation (right).	71
A.23	SysML v2 sequence diagrams: Miscellaneous (left) and Test execution (right).	72

List of Tables

3.1	Summary of representative related work along the five dimensions most relevant to this thesis.	11
4.1	Property graph node labels relevant to reconstruction. Each C++ node (the Cpp* labels) carries name, symbol, type, and lineNumber properties . .	14
4.2	Property graph relationship types, all populated by Renaissance from static analysis.	15
4.3	Agent skills composing the reconstruction pipeline.	16
5.1	Quantitative outcomes per pipeline stage on CPSCore.	32
6.1	H1 per-scenario results (interaction edges): precision / recall / F1 by condition. Scenarios S1a, S1b, S2, S3 all occur naturally in CPSCore (section 5.6.1).	34
6.2	H1 per-scenario results (execution-order LCS metric): precision / recall / F1 by condition, over the same four scenarios.	35
6.3	H1 macro-averaged results across all four scenarios, interaction edges and execution-order LCS metric.	35
6.4	H2 aggregate results: guided vs. full instrumentation (mean across S1a, S1b, S2, S3).	37
6.5	H2 qualitative results: mean blinded expert ratings (1–5 scale, $n = 6$ raters) for full vs. guided instrumentation, per scenario and rating dimension. . .	38
6.6	H3 holistic comparison: five conditions across all four scenarios, S1–S4. . .	39
6.7	Estimation-game answers, illustrating codebase scale on the second industrial system.	40
A.1	Generated component-interaction edges under condition C5 (intersection, the best-performing evaluated condition on interaction edges, section 6.2.2).	59

Acknowledgements

I would like to thank my daily supervisor at TNO-ESI, Dr. Rosilde Corvino, for her guidance, patience and consistent support throughout this project. Working within the Graph Development Intelligence (GDI) team alongside Navoneel has been challenging and rewarding. I am grateful for their collaboration and the many productive discussions we shared.

I also thank my academic supervisor at the University of Amsterdam, Dr. L. Thomas van Binsbergen, for his feedback on the research design and on the thesis writing throughout.

Thanks are owed to the six TNO-ESI engineers who took part in the blinded expert evaluation underpinning H2 — among them Navoneel, Bart and Joe — for their time reviewing the generated diagrams.

I would also like to thank the four engineers at TNO-ESI's industrial partner who took part in the hands-on session and interviews reported in section 6.5. Their time and candid feedback were invaluable in understanding how this line of work holds up on a real industrial-scale codebase.

I am also grateful to the maintainers of the open-source CPSCore framework [28], whose codebase made the case study in this thesis possible.

Finally, I thank my family and friends for their encouragement throughout the Master's programme.

Introduction

1.1 Context

Large-scale software-intensive systems maintained by industrial research organisations are typically mission-critical, long-lived and subject to continuous evolution [26]. Many operate in safety-critical or performance-sensitive domains such as medical imaging and industrial automation, where an incorrect change or a poorly informed architectural decision carries real technical and economic consequences. An accurate understanding of system behaviour is therefore needed before any refactoring, modernisation or architectural restructuring.

This understanding is captured by the *Interaction Model*: a formal, scenario-specific description of the message exchanges between components, most commonly expressed as sequence diagrams in Unified Modeling Language (UML) or, more recently, SysML v2. In practice, Interaction Models rarely stay current with the source code they describe. Complexity accumulates from evolving requirements, heterogeneous technologies and concurrency [26], and diagrams accurate when drawn drift out of sync as the system changes. Engineers are consequently left to reconstruct system behaviour directly from source code and limited runtime observations, a process that is time-consuming, error-prone and heavily dependent on individual expertise [26].

The problem is worse in event-driven systems, where asynchronous message passing, callbacks and indirect control flow obscure how components relate to one another [7]. Determining which components interact, in what order and under what conditions, is non-trivial, yet essential for identifying architectural bottlenecks, isolating faulty behaviour and assessing the impact of proposed changes, which is why industry needs accurate, scenario-specific interaction models reflecting how a system actually behaves under concrete execution scenarios [10].

A concrete illustration of this problem surfaced during this project: a real, previously-resolved defect in a large industrial C++ system (section 6.5.2). A stale proxy object was never freed because a reset call was missing from the connection-handling code, and the system silently stopped delivering change-notification events after a service restart.

The GDI project at Netherlands Organisation for Applied Scientific Research (TNO)-Embedded Systems Innovation (ESI) addresses this problem using graph-based program analysis

to support automatic extraction and maintenance of software architecture models. This thesis contributes a hybrid approach combining two complementary sources of evidence: a static call graph extracted from source code, and timestamped execution traces captured at runtime.

1.2 Problem Statement

Static analysis reveals call dependencies and structural relationships, giving a complete view of potential interactions, but it typically over-approximates behaviour and cannot distinguish interactions specific to one scenario from those occurring in every other scenario [26]. Trace-based analysis is scenario-specific by construction, but inherently incomplete [7]: in industrial systems, traces are often sparse or noisy due to limited logging, instrumentation overhead, or execution paths simply not exercised during recording.

Hybrid static-dynamic approaches have been proposed [7, 18], but existing methods primarily use static structure to organise or contextualise an already-observed trace (chapter 3); they do not actively infer interactions the trace never captured. No existing approach systematically combines the two directions at once: using static structure to recover what a trace missed, and the trace to trim an over-approximate static graph down to what is relevant to the scenario at hand. Closing that gap in both directions is the core problem this thesis addresses.

A further complication comes from the publish-subscribe indirection common in event-driven C* systems. *CPSCore*, the open-source framework used as the case study, routes inter-component communication through an Inter-Device Communication (IDC) abstraction layer backed by `boost::signals2` callbacks and Redis channels. A static call graph built from function call sites alone contains edges *into* the IDC layer, but cannot reveal which component receives each published packet, since the subscription topology is implicit in callback registrations rather than explicit in call sites. Recovering that missing structure is part of this thesis's contribution (section 4.5).

1.3 Research Questions

This thesis is guided by three research questions:

- RQ1** Does combining a static call graph with runtime execution traces reconstruct scenario-specific Interaction Models more accurately than either source alone?
- RQ2** Does guiding trace collection with the static call graph remove reconstruction noise without losing scenario coverage?
- RQ3** How well does a reconstructed SysML v2 Interaction Model consolidate static and dynamic evidence into a single artefact sufficient for system-behaviour comprehension?

Each is paired with a testable hypothesis, answered by a dedicated experiment (chapters 5 and 6):

- H1 (RQ1).** Combining static analysis and dynamic instrumentation improves reconstruction accuracy over either source alone.
- H2 (RQ2).** Code-graph-guided instrumentation improves scenario focus by suppressing irrelevant execution detail relative to full, unguided instrumentation.
- H3 (RQ3).** Reconstructed interaction models improve agent task performance on scenario comprehension, relative to reasoning from raw source code alone.

RQ1, RQ2 and RQ3 are answered empirically by the H1, H2 and H3 experiments on the CPSCore case study (chapters 5 and 6).

1.4 Approach

The approach runs as four phases (section 4.1): static call-graph analysis using Renaissance and Neo4j, which yields a reachable function set and a static dependency table (Phase 1); guided instrumentation at architecture-level call sites, producing a scenario-scoped runtime trace (Phase 2); a Neo4j-based reachability filter that scopes the trace to scenario-relevant events (Phase 3, condition C5); and SysML v2 generation from the filtered interaction set (Phase 4).

The approach is developed and validated on *CPSCore*, an open-source event-driven C++ framework, across four found-in-the-wild scenarios spanning a linear happy-path case to a multi-component indirect-dispatch case (chapters 5 and 6)

1.5 Outline

Chapter 2 introduces the technical concepts underpinning the approach: SysML v2 Interaction Models, property graphs, static call-graph analysis, execution tracing and scenario-scoped querying. Chapter 3 surveys existing work on program comprehension, architecture recovery and trace-based diagram synthesis. Chapter 4 details the design: the property graph schema, the trace-collection pipeline and the Neo4j-based reachability filter. Chapter 5 presents the implementation and results on the CPSCore case study. Chapter 6 analyses those results against the research questions and hypotheses, and discusses threats to validity. Chapter 7 summarises the contributions, discusses future work and concludes the thesis.

Background

This chapter provides the technical foundation for the approach in chapter 4: SysML v2 and its Interaction Model constructs, the property graph data model, the static and dynamic analysis techniques used, and how scenarios are scoped through graph queries. It is background rather than contribution.

2.1 SysML v2 and Interaction Models

The Object Management Group Systems Modeling Language version 2 (SysML v2) is a general-purpose modelling language for systems engineering, built on the Kernel Modeling Language (Kernel Modeling Language (KerML)) [23]. Unlike its predecessor (SysML v1, a profile of UML), SysML v2 defines its own textual and graphical notation, enabling model interchange via a standardised textual syntax.

2.1.1 Sequence Diagrams in SysML v2

Interaction Models in SysML v2 are expressed as *occurrence specifications*: ordered sequences of message occurrences between *parts* (instances of blocks in the system structure). The constructs used by the generator in this thesis (section 5.5) are:

part def :> InteractionScenario A named container for a scenario-specific interaction model, analogous to a UML interaction.

Lifeline A participant in the interaction: an instance of a structural block along which message occurrences are ordered.

SynchronousCall / ReturnMessage / AsynchronousMessage Message stereotypes, specialising a common Message definition, modelling a call, its return and an asynchronous send between two lifelines.

from / to References binding a message to its sending and receiving lifelines, giving each occurrence its position in the ordered sequence.

An example in SysML v2 textual notation, corresponding to scenario S1 (Aggregator notification chain) and following the same stereotype-based form the generator emits (section 5.5):

```

package AggregatorNotification {
  part def SynchronousCall :> Message { ... }

  part def S1_AggregatorChain :> InteractionScenario {
    part runner : Lifeline;
    part agg    : Lifeline;
    part log     : Lifeline;

    part m1 : SynchronousCall {
      ref from : Lifeline = runner;
      ref to   : Lifeline = agg;
      attribute :>> label = "1. getAll";
    }
    part m2 : SynchronousCall {
      ref from : Lifeline = runner;
      ref to   : Lifeline = log;
      attribute :>> label = "2. stream";
    }
    part m3 : SynchronousCall {
      ref from : Lifeline = runner;
      ref to   : Lifeline = log;
      attribute :>> label = "3. flush";
    }
    part m4 : SynchronousCall {
      ref from : Lifeline = runner;
      ref to   : Lifeline = log;
      attribute :>> label = "4. instance";
    }
  }
}

```

2.2 Property Graphs

A Property Graph is a directed, attributed multigraph in which both nodes and edges carry arbitrary key-value properties [2]. Formally, a Property Graph (PG) is $G = (V, E, \lambda, \mu)$, where V is the set of nodes; $\lambda(v) \in \mathcal{L}$ assigns a label from the label set $\mathcal{L}$ to each node $v \in V$; $E \subseteq V \times \mathcal{R} \times V$ is the set of directed edges, each carrying a relation type r from the relation-type set $\mathcal{R}$; and $\mu : (V \cup E) \times \mathcal{K} \rightarrow \mathcal{V}$ is a partial function assigning property values from value set $\mathcal{V}$ to nodes or edges, indexed by property keys from key set $\mathcal{K}$.

Neo4j is the most widely deployed Graph Database (GDB) implementing the PG model, queried via the Cypher declarative query language [22]. A PG was chosen over a relational or document store because it lets heterogeneous analysis artefacts (functions, files, modules and trace events) be traversed efficiently along typed relationships.

2.3 Static Call-Graph Analysis

A *call graph* is a directed graph in which nodes represent functions (or methods) and edges represent potential call relationships [14]: an edge (f, g) exists if and only if caller f may call callee g under some execution.

2.3.1 Clang and the Renaissance Semantic Extractor

Clang, the C++ frontend of LLVM, exposes an Abstract Syntax Tree (AST) traversal Application Programming Interface (API) that lets static analysis tools extract precise call-graph information [19]. Rather than driving Clang directly, this thesis uses *Renaissance*, a semantic-level C++ analyser already in use at TNO-ESI, to extract the static call graph and populate the property graph (section 4.2.1). Renaissance resolves the full AST, including template instantiations, overload candidates and macro expansions, and emits the schema described in section 4.2.

2.3.2 Limitations in Event-Driven Systems

Event-driven systems that route inter-component communication through callback registration or publish-subscribe indirection pose a specific problem for static call-graph construction. In CPSCore, subscriptions are registered via the IDC layer:

```
IDC::subscribeOnPacket(channelId, handlerSlot);
```

Standard AST traversal identifies the call to `subscribeOnPacket` but cannot resolve which component receives a packet published on the same channel, since publisher and subscriber are tied together only by a shared runtime identifier, not a call expression. The resulting static call graph contains edges *into* the IDC/Redis layer but lacks the cross-component routing information interaction model reconstruction needs. Section 4.5 addresses this by matching publisher and subscriber call sites on that shared identifier, the general mechanism behind CPSCore’s own IDC/Redis implementation (sections 2.6 and 5.1.3).

2.4 Execution Trace Analysis

An *execution trace* is a temporally ordered sequence of events recorded at runtime, typically by instrumenting the system under analysis with logging or profiling statements. In this thesis, traces take the form of timestamped log entries emitted at instrumented call sites rather than at function entry/exit (section 4.4), so a trace records that a specific cross-component call executed, not merely that a function was entered; each entry carries the caller and callee component identity and the name of the interaction invoked.

2.4.1 Challenges with Incomplete Traces

Practical execution traces are almost always incomplete: not every component is instrumented, recording overhead can drop log entries, and a single scenario recording covers only a subset of the system’s behaviour. These gaps are what motivate the hybrid approach. The static call graph supplies structural completeness, the trace supplies scenario specificity.

2.5 Scenario Scoping through Graph Queries

Reconstructing a diagram for a single scenario requires narrowing a large call graph down to the interactions that belong to it. Rather than a formal program-slicing procedure [30] over a program dependence graph [11], this thesis performs scenario scoping directly as a declarative graph query: a scenario's scope is expressed as a set of participating components (*e.g.* `Synchronization` calling into `Aggregation` and `Logging`), and a scenario-scoped Cypher query retains only the call edges whose caller and callee belong to those components, keeping what is relevant to the scenario and discarding interactions belonging to other, concurrently exercised behaviour.

The concrete query is described in section 4.6. It operates over the static property graph rather than a source-level program representation, and the same component-scope constraint is applied to the trace-derived sequence table.

2.6 Publish-Subscribe via Callback Registration

Many event-driven C++ systems decouple component communication through a publish-subscribe pattern, where subscriptions are registered at runtime via callback slots rather than direct function calls, making the topology implicit in the source and unrecoverable from call sites alone (section 2.3). CPSCore implements this through its IDC layer, which delegates to a pluggable `INetworkLayer` backed by `boost::signals2` and Redis channels (section 5.1.3). Recovering the subscription topology means matching publisher and subscriber call sites by their shared channel identifier, the approach taken in section 4.5.

Related Work

This chapter surveys existing work relevant to this thesis. It is organised thematically: (i) trace-based interaction model extraction, (ii) scenario-driven and functionality-centric analysis, (iii) behavioural pattern mining, (iv) static analysis for architecture recovery (including the specific problem of publish-subscribe dispatch), (v) hybrid static-dynamic approaches and (vi) trace visualisation for program comprehension. For each area, the key techniques are described and their limitations with respect to the problem addressed in this thesis are identified. Representative studies for each area are summarised in table 3.1.

The role of related work here is threefold. First, it grounds the approach in prior techniques. Second, it identifies the gap that motivates this thesis. Third, it establishes the baseline against which the results are compared.

3.1 Trace-Based Interaction Model Extraction

A significant body of work reconstructs UML sequence or interaction diagrams directly from execution traces. Early work by Taniguchi et al. [27] and Grati, Sahraoui and Poulin [13] extracts sequence diagrams from Java execution traces. This demonstrates that runtime information can be mapped onto object interactions. Ziadi et al. [31] propose a fully dynamic reconstruction approach that relies exclusively on traces with no static analysis support. Hamou-Lhadj and Lethbridge [15] instead target scale. They summarise large execution traces into compact UML sequence diagrams by identifying and merging recurring interaction patterns. Baidada, Bouziane and Jakimi [3] take a similar approach. They reconstruct higher-level sequence diagrams by merging information from multiple execution traces and modelling the result as Petri nets. Briand *et al.* [4] use scenario-based testing to drive trace collection and then synthesise sequence diagrams from the resulting traces. This is scenario-specific but requires a scenario to be manually specified and exercised up front. Shang et al. [25] show that even ordinary system logs (never intended as execution traces) can be mined into architecture-level interaction models. However, this comes at the cost of coarse granularity.

3.1.1 Limitations

All trace-based approaches share the same structural limitation: they can only reconstruct interactions that were actually observed. Missing events, low logging verbosity and execution paths that were never exercised during recording all show up as gaps in the reconstructed diagram. In an industrial system with partial instrumentation, that is the common case rather than the exception.

3.2 Scenario-Driven and Functionality-Centric Analysis

A separate line of work treats the *scenario* itself as the organising abstraction rather than the raw trace. Egyed [10] introduces a scenario-driven approach to trace dependency analysis. This approach explicitly links scenarios to execution footprints and source-code artefacts. It demonstrates that scenarios can act as a unifying concept across requirements, traces and code. Briand *et al.* [4], already noted above, formalise the mapping between runtime events and interaction diagrams around execution scenarios in distributed systems. Despite recognising the value of scenario scoping, none of this work asks whether static dependencies could be used to *complete* a scenario-specific interaction flow when the corresponding trace is incomplete or noisy. The scenario bounds what is observed but does nothing to fill in what was not observed.

3.3 Behavioural Pattern Mining and Model Inference

A third line of research infers higher-level behavioural abstractions from execution data rather than concrete interaction flows. Lo *et al.* [20] mine quantified temporal rules from execution traces to extract recurring behavioural patterns across runs. Gabel and Su [12] similarly mine general temporal properties expressed as finite-state automata over method invocations. They do so fully automatically from dynamic traces without requiring a pre-specified property to check against. Walkinshaw *et al.* [29] reconstruct state-based behavioural models (state machines) from software executions. All three approaches infer generic behavioural structure purely from dynamic traces. They use no static structural component and have no explicit grounding in a specific functionality scenario. The resulting models are rules, properties, or state machines rather than concrete sequence diagrams. This makes them a poor fit for understanding one particular execution flow.

3.4 Static Analysis for Architecture Recovery

Static analysis approaches extract architectural information directly from source code without executing the system. Formal call-graph construction techniques for C++ (including class hierarchy analysis and points-to analysis) are surveyed in Grove *et al.* [14]. These techniques produce over-approximate graphs that include every possible interaction but cannot distinguish which interactions occur in a given scenario.

Ducasse and Lanza [9] propose the *class blueprint*, a polymetric visualisation that overlays software metrics on a class's internal structure (methods, attributes, and their in-

vocation relationships) to support a first, structural understanding of an unfamiliar class. This line of work targets comprehension at the class level rather than component-level interaction recovery and does not target C++ or scenario-specific analysis. Component-level dependency recovery is instead the focus of tools such as *Understand and Structure*¹⁰¹, which produce rich static dependency graphs but do not generate scenario-specific sequence diagrams.

3.4.1 Handling Event-Driven Dispatch

Static analysis of event-driven systems with publish-subscribe topologies is particularly challenging. The subscription relationship between a publisher and a handler is usually established indirectly (for example, via a topic string or a registration callback) rather than through a direct call site. Santos, Cunha and Macedo [24] address this for Robot Operating System (ROS), where inter-component communication is mediated by a topic broker. Their approach statically extracts the ROS computation graph by matching publisher and subscriber declarations by topic name to reconstruct the communication topology. This is a strategy closely related to the publish-subscribe topology resolution developed in this thesis for CPSCore’s IDC/Redis layer (section 5.1.3).

3.5 Hybrid Static and Dynamic Analysis for Behaviour Recovery

Hybrid approaches combining static and dynamic analysis aim to overcome the completeness versus scenario-specificity trade-off directly. Systä [26] combines static and dynamic views to support program comprehension. This work shows that static structure can contextualise runtime behaviour. Hassan and Holt [16] combine static and dynamic information to predict change propagation. Their work demonstrates the benefit of integrating multiple analysis sources for a different task.

Labiche, Kolbah and Mehrfard [18] propose a hybrid static-dynamic approach for reverse-engineering scenario diagrams. They combine lightweight dynamic instrumentation with static control-flow analysis to generate UML sequence diagrams from Java systems. Critically, static analysis in their approach is used to *structure and contextualise observed executions*. It is not used to infer interactions that were never observed. This is the same gap identified in chapter 1. Cornelissen et al. [7] conduct a systematic review of dynamic analysis for program comprehension and independently identify the lack of integration with static information as a major open problem. More recently, Abdelmalek, Khriess and Jakimi [1] survey and categorise approaches for reverse-engineering behavioural UML diagrams, grouping them into static, dynamic and hybrid analysis, and themselves propose a hybrid approach that uses static analysis to identify system entry points and guide dynamic instrumentation. Neither their survey nor their proposed approach addresses C++ or publish-subscribe indirection specifically, and their hybrid technique still requires exercising the target behaviour at runtime rather than reconstructing scenario-specific models from incomplete traces.

Overall, hybrid analysis is widely acknowledged as beneficial. However, existing approaches primarily enrich or organise already-observed runtime behaviour. They do not systematically reason about incomplete traces or reconstruct unobserved component interactions within a bounded execution scenario.

3.6 Trace Visualisation for Program Comprehension

A further body of work improves program comprehension by presenting execution traces at varying levels of abstraction, rather than by reconstructing a formal model. Cornelissen, Zaidman and Deursen [6] and Cornelissen et al. [7] study trace visualisation techniques and their effectiveness for understanding large execution traces. Hamou-Lhadj and Lethbridge [15] propose trace summarisation techniques to reduce noise and focus attention on relevant behaviour, and De Pauw et al. [8] introduce visual representations of object-oriented program execution aimed at scalability. These techniques improve trace readability but, again, do not aim to reconstruct a formal interaction model or to integrate static analysis to complete missing behaviour.

3.7 Summary and Positioning

Table 3.1 summarises a representative selection of the approaches reviewed above along the dimensions most relevant to this thesis.

Table 3.1: Summary of representative related work along the five dimensions most relevant to this thesis.

Approach	Static	Trace	Hybrid	Pub-sub	SysML
Hamou-Lhadj & Lethbridge [15]	×	✓	×	×	×
Briand <i>et al.</i> [4]	×	✓	×	×	×
Shang <i>et al.</i> [25]	×	✓	×	×	×
Egyed [10]	×	✓	×	×	×
Lo <i>et al.</i> [20]	×	✓	×	×	×
Gabel & Su [12]	×	✓	×	×	×
Ducassee & Lanza [9]	✓	×	×	×	×
Santos <i>et al.</i> [24]	✓	×	×	✓	×
Systä [26]	✓	✓	✓	×	×
Labiche <i>et al.</i> [18]	✓	✓	✓	×	×
This thesis	✓	✓	✓	✓	✓

No existing approach addresses all five dimensions simultaneously. In particular, three things are missing from prior work. First, none combines static call-graph evidence with runtime traces for SysML v2 interaction model reconstruction while keeping track of which source each interaction came from. Second, none applies scenario-scoped graph querying to focus the result on a single scenario. Third, none targets SysML v2 as the output formalism. This gap constitutes the primary research contribution of this thesis.

Methods

This chapter describes the design of the hybrid reconstruction pipeline: the property graph schema that stores the static artefacts, the separate trace-collection pipeline, the agent skill architecture that can optionally be used to orchestrate extraction and generation, the pattern-based instrumentation tool used for trace collection, the scenario-scoped querying and Neo4j-based reachability filter that produces scenario-specific interaction models.

4.1 Pipeline Overview

The pipeline takes a C++ source tree and, optionally, a set of timestamped execution traces, and produces SysML v2 sequence diagrams scoped to a user-specified scenario. It runs as four phases (fig. 4.1). All four phases follow a fixed, deterministic rule set given their inputs.

1. **Phase 1 — Static Analysis (Renaissance)** (section 5.2). Renaissance analyses the C++ source and populates the Neo4j graph with 11 node labels and 11 relationship types (section 4.2); the CPSCore graph contains 67,104 nodes. Two Cypher queries then run against the live graph: a scenario-scoped forward traversal (`CppCalls*1..8` from the scenario's entry-point functions) producing the *reachable function set* consumed by Phase 3 (sections 4.6.1 and 4.6.2), and a global cross-component query producing the architectural interface map identifying call sites eligible for instrumentation (section 4.4). The scenario-scoped subgraph is also flattened into an ordered CSV table (condition C2 static edge set, section 6.2.2) via `get-interface-dependency-table` and `get-sequence-dependency-table`.
2. **Phase 2 — Guided Instrumentation** (add-runtime-instrumentation, section 4.4). Pattern-based code transformation injects trace probes at the call sites identified in Phase 1; the instrumented binary is rebuilt and run, producing `trace_slice.txt`.
3. **Phase 3 — C5: Neo4j-Based Reachability Filter** (synthesis, section 4.6.2). `trace_slice.txt` is filtered to events whose caller is in the Phase 1 reachable function set, removing setup-phase boilerplate and producing `interactions.json` and `sequence.json`.

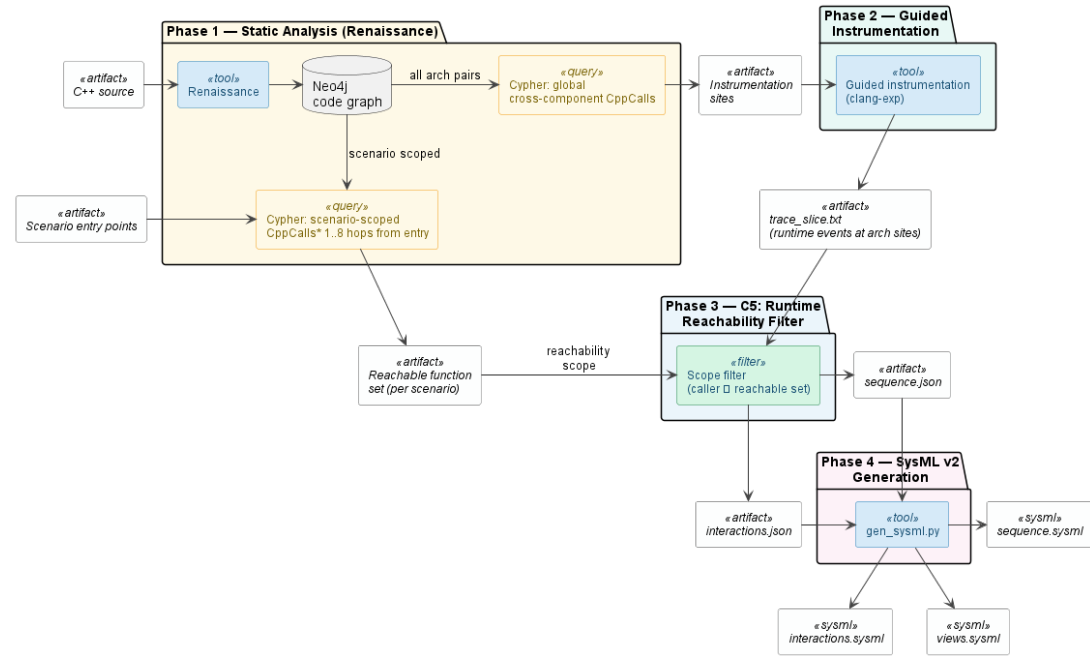


Figure 4.1: The concrete C5 pipeline as run for the H1 experiments, with the actual tools and artefacts behind each stage: Renaissance populating the Neo4j code graph and computing a scenario-scoped reachable function set via `CppCalls*1..8` traversal (Phase 1); guided instrumentation producing `trace_slice.txt` at architecture-level call sites (Phase 2); the Neo4j-based reachability filter scoping `trace_slice.txt` to callers in the reachable function set and writing `interactions.json` and `sequence.json` (Phase 3, condition C5); and `gen_sysml.py` rendering the final SysML v2 output (Phase 4, sections 4.6.2 and 4.8).

4. **Phase 4 — SysML v2 Generation** (`draw-sysml-sequence-model`, `draw-sysml-structural-model`, section 4.8). `interactions.json` and `sequence.json` are projected into SysML v2 textual notation.

Two further skills, `draw-sequential-mermaid-general` and `draw-structural-mermaid`, offer a Mermaid-syntax visualisation of the same reconstruction; they read the Phase 1 dependency tables directly and are not part of this four-phase pipeline, since they bypass the reachability filter rather than consuming its output.

Each skill is stateless and read-only with respect to the graph, except the instrumentation skill, which modifies source files. Every skill accepts a natural-language prompt and returns a structured artefact (CSV, Cypher query, or model file), which allows the chain to be reused on a new codebase without code changes.

4.2 Property Graph Schema

4.2.1 Design Decision

A PG in Neo4j was chosen for the static knowledge store over a relational database, a document store, and an in-memory graph library. The pipeline traverses `function`→`file`→`folder` paths, `function`→`function call chains`, and `class`→`function containment` within the same query. The static graph can also be re-derived or extended without disturbing the separate trace pipeline, since static edges are typed and distinguishable from anything added later, and Cypher’s pattern-matching syntax maps directly onto the `text-to-cypher` skill (section 4.3): a natural-language scenario query becomes a graph pattern with little translation. An in-memory graph library would satisfy traversal alone, and a document store neither traversal nor declarative querying.

4.2.2 Node Types

Table 4.1 lists the 11 node labels emitted by Renaissance.

Table 4.1: Property graph node labels relevant to reconstruction. Each C++ node (the Cpp* labels) carries `name`, `symbol`, `type`, and `lineNumber` properties

Label	Semantics
CppDeclaration	Class, struct, enum, or typedef declaration
CppForwardDeclaration	Forward declaration of a class or struct
CppFunctionDeclaration	Function prototype (header)
CppFunctionDefinition	Function body (implementation)
CppMacroDefinition	Preprocessor macro definition
HeaderFile	.h/.hpp file
SourceFile	.cpp/.c file
OtherFile	Non-C++ file tracked in the project
Folder	Directory in the source tree
Symbol	Demangled symbol name (linked to declarations via Name edges)
ProjectCOrCpp	CMake compilation unit

Renaissance additionally emits four build-system labels (`DynamicLinkLibrary`, `Executable`, `ProjectPropertySheet`, `Solution`) omitted from the table: they carry no interaction-reconstruction information and the pipeline never queries them. The graph holds static artefacts only.

4.2.3 Relationship Types

Table 4.2 lists the 11 relationship types populated by Renaissance.

The static `CppCalls` edges and the trace-derived sequence table are kept as two independent edge sets and compared only at evaluation time (section 4.6): a `CppCalls` edge says a call *may* happen, since it is read from source code listing every call that could occur, while a trace record says a call *did* happen, recorded from a real run. The static edges therefore tend to over-count and the trace tends to under-count.

Table 4.2: Property graph relationship types, all populated by Renaissance from static analysis.

Type	Source	Target	Semantics
CppCalls	FunctionDef	FunctionDecl/Def	Direct call site
CppContains	Declaration	Declaration/FunctionDecl/Def	Lexical containment
CppImplements	FunctionDef	FunctionDecl/Def	Declaration-definition link
CppInherits	Declaration	Declaration/ForwardDecl	Class inheritance
CppAlias	Declaration	Declaration	Type alias
CppUses	File	Declaration/ForwardDecl/Macro	Symbol usage in file
Include	File	File	Preprocessor include
Name	Cpp* node	Symbol	Name binding
ParentFolder	File/Folder	Folder	Directory hierarchy
ProjectCompilesNested	ProjectCOrCpp	File	Compilation membership
Source	Cpp* node	File	Source-location back-reference

4.3 Agent Skill Architecture

The four-phase pipeline is also exposed as twelve composable agent skills invocable via natural-language prompt, providing a practitioner-facing interactive interface for applying the pipeline to a new codebase. The H₁, H₂, and H₃ evaluations run the pipeline through the underlying Python scripts directly, not through the skill layer. The skill layer was chosen over a monolithic implementation for three reasons:

- **Codebase agnosticism.** Skills query the graph schema dynamically (`get-neo4j-schema`) rather than hardcoding node labels, so switching target codebases requires only re-running Renaissance.
- **Composability.** Each skill produces an intermediate artefact (a CSV, a Cypher query, a model file) that an engineer can inspect, edit, or override before it feeds the next skill.
- **Scenario flexibility.** The scope of a reconstruction can be expressed as a natural-language query (“show me the interaction between Synchronization and Aggregation when a runner timeout occurs”) rather than a hardcoded configuration, with `text-to-cypher` translating it into the appropriate Cypher pattern at invocation time.

Table 4.3 summarises all twelve skills, their inputs, outputs, and role in the pipeline. Deterministic skills (instrumentation, synthesis, and diagram generation) follow a fixed rule set given their inputs; query-generation skills (`text-to-cypher`, `cypher-to-output`, `get-cypher`) use an LLM and may produce varied output for the same input.

Table 4.3: Agent skills composing the reconstruction pipeline.

Skill	Input	Output	Role
get-neo4j-schema	Neo4j connection	graph_schema.txt	Schema introspection
text-to-cypher	Schema + NL query	Cypher query	Query generation
cypher-to-output	Cypher query	Human-readable answer	Query execution + explanation
get-interface-dependency-table	Neo4j graph	Interface CSV	Structural extraction
get-sequence-dependency-table	Neo4j graph	Sequence CSV	Sequence (CppCalls) extraction
add-runtime-instrumentation	Source files + patterns	Instrumented source + Trace CSV	Probe injection via csp_matcher, rebuild, run, and trace collection
synthesis	Static table + trace (CSV or JSON)	interactions.json + sequence.json	Runtime trace filtered by static-graph reachability scope
draw-sysml-sequence-model	Sequence CSV or sequence.json	.sysml file	Sequence diagram generation
draw-sysml-structural-model	Interface CSV or interactions.json	.sysml file	Structural diagram generation
draw-sequential-mermaid-general	Sequence CSV	.mmd file	Mermaid sequence visualisation
draw-structural-mermaid	Interface CSV	.mmd file	Mermaid dependency visualisation
get-cypher	NL query	Cypher (no execution)	Standalone query authoring

4.4 Pattern-Based Instrumentation (csp_matcher)

Automated insertion of trace probes into an existing C++ codebase is a prerequisite for runtime trace collection: manual instrumentation is error-prone and does not scale, and a preliminary experiment with an LLM alone proved brittle, since the model could not reliably identify every call site of interest and tended to over-instrument. The `csp_matcher`

tool, developed alongside Dr. Rosilde Corvino as part of this thesis, addresses this with AST-level pattern matching and code transformation.

4.4.1 Design

csp_matcher is a Clang LibTooling-based tool that parses a user-provided pattern (near-normal C++ with hole variables) into a Clang AST, translates it into ASTMatcher DSL expressions, runs the generated matchers against target source files, and optionally rewrites matched code using a template that back-references captured holes.

Patterns use two kinds of hole variables:

\$name Matches any single expression or identifier.

\$name Matches a statement list or parameter list.

Two context variables are automatically injected for every match: `$callerFunc` (the enclosing function name) and `$callerClass` (the enclosing class name, empty at file scope).

Example: wrap every `getAll` call with a trace statement:

```
find:    $agg.getAll<$T>()
replace: (std::fprintf(stderr, "[TRACE] %s %s::%s\n",
        cspTraceTimestamp().c_str(),
        "$callerClass", "$callerFunc"),
        $agg.getAll<$T>())
```

This is preferable to manual macro insertion in three respects: the transformation is declarative, so a rules file states *what* to match and replace rather than *how* to walk the AST; it is reproducible, since re-running the same rules file produces the same edits; and it is reversible, via a corresponding `remove:` rule (section 4.4.3). Filter functions, compiled at runtime from plain C++, additionally allow selective matching, *e.g.* excluding test-internal call sites.

4.4.2 Integration with the Pipeline

The call-site patterns instrumented are not chosen ad hoc: they come from the global, scenario-independent Cypher query over all cross-component `CppCalls` edges (section 4.1), which yields the codebase’s architectural interface map – every call site where one component invokes another. Only call sites appearing in that map are eligible for a probe.

The `add-runtime-instrumentation` skill invokes `csp_matcher` with a rules file (`add_tracing.txt`) and a filter definitions file (`tracing_filters.cpp`), following a two-pronged approach. First, call-site wrapping: target call expressions such as `Aggregator::getAll`, `CPSLogger::flush`, and `EnumMap::convert` are wrapped with a comma-operator trace emission. Second, `CPSLOG` macro patching: the `CPSLOG(level)` macro in `CPSLogger.h` is patched to emit a `[TRACE]` line on every logging call, covering all `RAIILogStream::stream` call sites regardless of translation unit. The macro patch is applied only *after* call-site wrapping is complete, since `csp_matcher` parses target

files using a pattern preamble that includes `CPSLogger.h`, which must be in its clean, unpatched state when the matcher runs.

Once the probes are in place, the same skill rebuilds and runs the target (the CPSCore test suite, or a scenario-specific harness) and captures the emitted `[TRACE]` lines from `stderr`. This log is normalised into the same schema as the static dependency table (section 4.1) so the two can later be compared edge-for-edge: it does not pass through `get-sequence-dependency-table`, which extracts static `CppCalls` edges from the property graph rather than reading trace output; the normalised trace log instead feeds the evidence combination and synthesis stages directly (sections 4.6 and 4.6.2).

4.4.3 Reversibility via Filter Functions

Instrumentation removal is handled by a single generic rule:

```
find:    ($any, $orig)
filter:  is_trace_remove
replace: $orig
```

This matches every comma expression in the codebase, but the `is_trace_remove` filter rejects any match whose left operand does not contain the `[TRACE]` marker string:

```
bool is_trace_remove(int n, const char* const* ns,
                    const char* const* vs) {
    const char* text = get_hole(n, ns, vs, "any");
    return text != NULL && strstr(text, "[TRACE]") != NULL;
}
```

Only trace-injected comma expressions are stripped; pre-existing comma expressions are left untouched, so applying `remove_tracing.txt` restores the source tree to its pre-instrumentation state byte-for-byte. `csp_matcher` was validated by an automated test suite of 130 cases across seven suites (section A.2).

4.5 Macro Resolution for Publish-Subscribe Systems

The CPSCore IDC layer (section 5.1.3) routes messages through `boost::signals2` callbacks: a publisher calls `sendPacket(id, payload)` and a subscriber registers a handler via `subscribeOnPacket(id, slot)`. No direct call site connects the two, so a raw static call graph has no edge between the publisher and the subscriber's handler.

Although this interaction appears in the runtime trace, the Neo4j-based reachability filter (Phase 3) decides what to keep by traversing `CppCalls` edges in the Neo4j graph. Without a synthetic edge connecting publisher to subscriber, the subscriber's handler falls outside the reachable function set and the filter removes it. Macro resolution therefore inserts a `PubSubRoute` edge for every matched `(sendPacket, subscribeOnPacket)` pair sharing the same `id`, ensuring the reachability filter retains pub-sub interactions.

4.6 Scenario Scoping

4.6.1 Component Constraint Filter

The static and dynamic evidence are each projected to a common schema of directed, component-level interaction edges keyed by (ClientComponent, ServerComponent, EventName). The static edge set is obtained with a plain Cypher query:

```
MATCH (caller:CppFunctionDefinition)-[:Source]->(f)-[:ParentFolder]->(folder)
MATCH (caller)-[:CppCalls]->(callee)
WHERE last(split(folder.name, '/')) = $component
RETURN caller, callee
```

The dynamic edge set is read from the real runtime trace and normalised into the same schema.

Scenario scoping is a component-level filter applied around a forward call-graph traversal, rather than the program-dependence-graph slice described in section 2.5. For each scenario, the traversal starts from the scenario's declared entry-point functions – the specific production methods the exercising test calls, *e.g.* `SynchronizedRunner.runSynchronized` and `SynchronizedRunnerMaster.runStage` for `S1a/S1b`, or all four of `AggregatableRunner.runAllStages`, `AggregatableRunner.notifyAggregationOnUpdate`, `MultiThreadingScheduler.schedule` and `MultiThreadingScheduler.run` for `S3` – and follows `CppCalls` edges outward, combining a direct entry→callee pattern with a declaration-to-implementation bridge (`CppImplements`) so that a call made through an interface still resolves to its concrete target:

```
MATCH (entry {type:'CppFunctionDefinition'})
WHERE entry.symbol = $method AND entry.name CONTAINS $cls
WITH entry LIMIT 1
MATCH (entry)-[:Source]->(ef)
WITH entry, ef
CALL {
  WITH entry, ef
  MATCH (entry)-[:CppCalls]->(decl)<-[:CppImplements]-(step_def)-[:CppCalls]->(callee)
  MATCH (callee)-[:Source]->(cf)
  WHERE ef.name <> cf.name
  RETURN callee.symbol AS sym, cf.name AS callee_file
  UNION
  WITH entry, ef
  MATCH (entry)-[:CppCalls]->(callee)
  MATCH (callee)-[:Source]->(cf)
  WHERE ef.name <> cf.name
  RETURN callee.symbol AS sym, cf.name AS callee_file
}
RETURN DISTINCT sym, callee_file, ef.name AS entry_file
```

The query runs once per declared entry point and the results are pooled. A bounded variable-length variant of the same traversal (`CppCalls*1..6`) additionally ranks each call by its minimum distance from the entry point, giving the depth-based ordering used for the sequence metric.

The traversal itself returns every cross-file call reachable from an entry point; the scenario's component allowlist is then applied as a filter over the results, keeping only

edges whose caller and callee both resolve to one of the scenario’s declared components and discarding same-component calls. The same allowlist filters the trace-derived table by row, so both evidence sources end up scoped to the same component set even though the queries that produce them are otherwise independent (section 4.4).

Formal definition of scope. Let E_s be the static interaction edge set and E_t the trace-derived edge set, with $\text{src}(e)$ and $\text{tgt}(e)$ an edge’s caller and callee. Let $\sigma = (\text{primary}, \text{targets}, \text{functions})$ be the scenario scope. The component-scoped edge set is:

$$E_\sigma = \{e \in E_s \cup E_t \mid \text{src}(e) \in \sigma.\text{primary} \wedge \text{tgt}(e) \in \sigma.\text{targets}\}$$

that is, the union restricted to edges whose caller and callee both belong to the scenario’s declared component set. This is a single-hop edge filter, which suffices for CPSCore, where cross-component calls are shallow (typically one hop through the IDC layer); multi-hop scoping is a straightforward extension via Cypher’s variable-length path syntax.

4.6.2 Neo4j-Based Reachability Filter

The Neo4j-based reachability filter scopes the dynamic evidence to functions reachable from the scenario’s entry points. For condition C5, the filter keeps only events from `trace_slice.txt` whose `ClientFunction` is reachable from the scenario’s `SCENARIO_ENTRY` functions via a bounded `CppCalls*0..8` traversal in the Renaissance Neo4j graph. This removes setup-phase boilerplate — events that fire at guided instrumentation sites before the scenario logic starts — while retaining every event triggered within the scenario’s actual execution scope.

Condition C6 applies the same filter to `full_trace.txt`, confirming the filter is sufficient regardless of instrumentation strategy. Condition C4 reads `trace_slice.txt` without the filter, providing the ablation point that isolates the filter’s contribution. Conditions C7–C9 supply the artefacts from C2, C3, or C4 as a candidate list to a two-prompt LLM chain that post-processes them; no set algebra or reachability query is involved in those conditions.

The output of the deterministic path (C4 or C5) is written to `interactions.json` (deduplicated (`source`, `target`, `interaction`) triples) and `sequence.json` (the same interactions in execution order with repetitions preserved). These files are consumed by SysML generation (section 4.8).

4.7 Ground-Truth Derivation

Ground-truth construction begins with manual code inspection. The architect reads the C++ source for each scenario, identifies the cross-component call sites at the architectural boundary, and draws structural and sequence diagrams in Excalidraw (section A.3). These hand-drawn diagrams are the primary reference artefact; they are not derived from any pipeline output.

The Excalidraw diagrams feed two downstream artefacts. First, the identified call sites are passed to Renaissance and `clang-exp` to configure guided instrumentation, so

probes are placed exactly at the architecture-level boundaries visible in the diagrams. Second, the component pairs and entry-point functions visible in the diagrams are encoded as filters in `arch_filters.py`: a REACHABILITY component-pair allowlist and a SCENARIO_ENTRY entry-point set.

When the instrumented test is run, it emits `trace_slice.txt`. `derive_gt_arch.py` reads this trace, applies both filters from `arch_filters.py`, and deduplicates to `gt_interactions.json` and `gt_sequence.json`. The ground truth is therefore fully determined by the architect's manual inspection: the same Excalidraw diagrams that configure instrumentation also define the filtering rules, making the derivation independent of the Neo4j-based reachability filter and directly traceable to the hand-drawn reference.

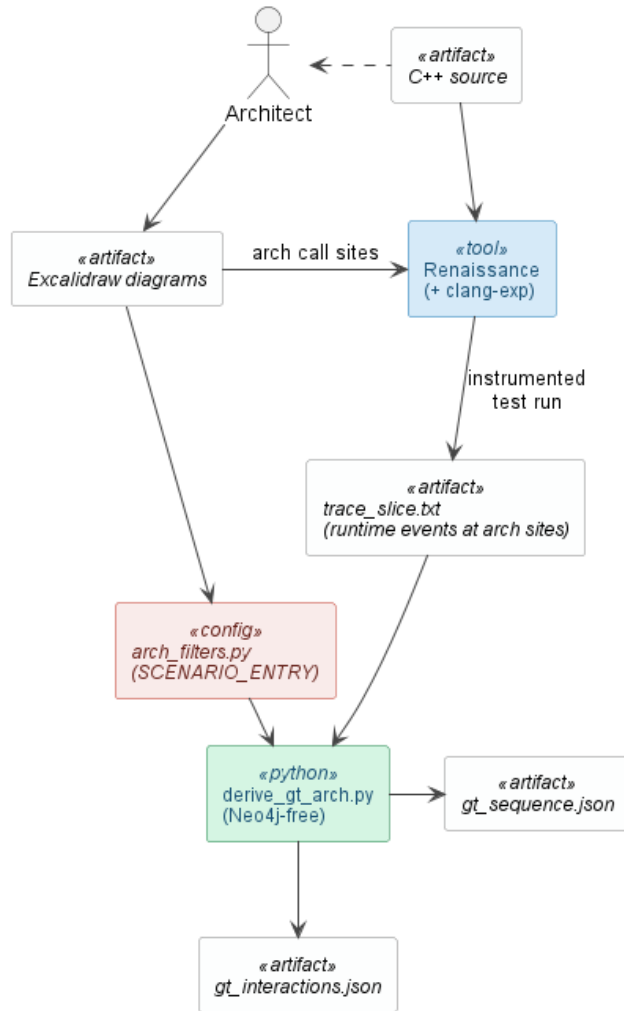


Figure 4.2: Ground-truth derivation pipeline. An architect manually inspects the C++ source and draws Excalidraw structural and sequence diagrams for each scenario (section A.3). These diagrams serve two roles: they identify the architecture-level call sites passed to Renaissance and clang-exp for guided instrumentation, and they define the SCENARIO_ENTRY entry-point and REACHABILITY component-pair filters encoded in arch_filters.py. The instrumented test run produces trace_slice.txt; derive_gt_arch.py then applies both filters and deduplicates to gt_interactions.json and gt_sequence.json.

4.8 SysML v2 Generation

The final stage projects the reachability-filtered output, `interactions.json` and `sequence.json` (section 4.6.2), into SysML v2 textual notation, using the SysML v2 constructs described in section 2.1. The `draw-sysml-structural-model` skill reads `interactions.json`: each unique component becomes a `part def`, each interaction it sends or receives becomes a typed port on that part, and each `(source, target, interaction)` triple becomes a `connect` statement between the corresponding client and server ports. The `draw-sysml-sequence-model` skill reads `sequence.json`: each unique component becomes a lifeline (`part def`), and each step becomes an ordered, labelled `SynchronousCall` between the corresponding lifelines, with the step number preserving the execution order recorded by synthesis.

Component names are derived algorithmically from file paths, not from a hardcoded mapping table, using a path-to-component resolution function that handles `/src/`, `/include/`, `/extern/`, and `/tests/` prefixes, CamelCase normalisation, and `Impl` suffix stripping. This keeps the generator codebase-agnostic: the same script produces meaningful lifeline names on CPSCore or any other source tree following standard C++ project layout conventions.

Implementation

This chapter describes how the design of chapter 4 was realised: the representative system used for development, each implementation stage with its code artefacts, and the concrete measurements obtained at each stage.

5.1 Representative System: CPSCore

The pipeline was developed and validated on *CPSCore* [28], an open-source modular C++ framework for cyber-physical systems originally built for the uavAP autopilot project. It was chosen for three reasons: it is freely available, it uses a real event-driven publish-subscribe architecture, and its test suite (1,425 assertions across 61 test cases) provides a ground truth for behavioural verification.

5.1.1 Scale and Dispatch Characteristics

CPSCore is a small codebase: 154 source and header files, roughly 13,000 lines (3,800 in .cpp files, 9,200 in headers). It relies heavily on template-based static polymorphism (≈ 470 occurrences) and declares only 32 virtual method signatures across 12 files, all confined to a single module (Utilities: `INetworkLayer`, `ITransportLayer`, `IPublisherImpl`, `IScheduler`) or to per-module configuration boundaries (`IConfigurableObject`, `IRunnableObject`). No cross-module call resolves through a virtual table; inter-module calls go through concrete class references or `boost::signals2` slot connections.

This has a direct consequence for H_1 (section 6.2.2): since no *direct* cross-module call resolves through a virtual table, the static call graph is reachability-complete with respect to the trace on every genuine interaction reached through a direct call – C_2 has zero recall gap on any true edge in S_1 , S_2 and S_4 (section 6.2.3). S_3 is the one exception: one of its genuine edges is reached only through a `std::function`-wrapped thread dispatch, a form of indirection distinct from virtual dispatch that the static graph still cannot resolve, giving C_2 a real recall gap there (section 6.2.3). Neither claim is a strict-subset claim ($C_3 \subseteq C_2$), and neither extends to false positives: on S_1 and S_4 , C_2 and C_3 each carry their own false-positive edge that the other source does not (C_2 's from an unfiltered static over-approximation, C_3 's from an unfiltered test-fixture-setup call in

the trace), so C4’s false-positive count is strictly the sum of both, never a subset of either (section 6.2.3). A codebase with pervasive cross-module virtual dispatch or dynamic plugin loading would likely give static analysis a recall gap on a wider range of scenarios than the single instance (`std::function` dispatch) CPSCore exhibits here.

5.1.2 CPSCore Architecture

CPSCore is organised into five modules: **Aggregation** (component wiring and lifecycle), **Configuration** (property mapping), **Synchronization** (run-stage scheduling), **Logging** (CPILogger, RAIILogStream), and **Utilities** (schedulers, data presentation, signal handling, Inter-Process Communication (IPC), IDC). External dependencies are Eigen3, Boost.System, `cpp_redis/tacopie`, and Catch2.

5.1.3 Publish-Subscribe via IDC and Redis

Inter-component communication runs through the IDC abstraction layer, whose two operations (`subscribeOnPacket`, `sendPacket`) delegate to a pluggable `INetworkLayer`. The default `RedisNetworkLayer` maps channel identifiers to `RedisSubscriber/RedisPublisher` objects backed by `cpp_redis`, wired through a `boost::signals2 slot` (`onPacket_.connect(slot)`) that fires when a Redis message arrives – the concrete instance of the call-site blind spot that section 4.5 resolves.

5.2 Phase 1 — Static Analysis

5.2.1 Renaissance to Neo4j

CPSCore’s source tree was analysed by Renaissance, which resolves the full AST (template instantiations, overload candidates, macro expansions) and emits the property graph schema in tables 4.1 and 4.2: 11 node labels and 11 relationship types, all from static analysis, populating a graph of 67,104 nodes. Runtime traces are processed separately into a sequence table and combined with the static edge set only at evaluation time (section 4.6).

5.2.2 Schema Introspection Skill

The `get-neo4j-schema` skill runs three Cypher statements against the live graph to produce a machine-readable schema summary for downstream skills:

```
-- Node labels and properties
CALL db.schema.nodeTypeProperties()
YIELD nodeType, propertyName, propertyTypes
RETURN nodeType, propertyName, propertyTypes;

-- Relationship types and properties
CALL db.schema.relTypeProperties()
YIELD relType, propertyName, propertyTypes
RETURN relType, propertyName, propertyTypes;
```

```
-- Relationship signatures (which label pairs each edge connects)
MATCH (s)-[r]->(t)
RETURN DISTINCT labels(s) AS fromLabels,
                  type(r)   AS relType,
                  labels(t) AS toLabels;
```

5.2.3 Scenario-Scoped Query

For a given scenario, `text-to-cypher` translates the scenario description into a schema-aware Cypher query and `cypher-to-output` executes it against the live graph, returning the scenario-scoped subgraph consumed by Phase 1. For `S1a/S1b`, which share a component scope (“Using agent skills, show me Synchronization calling into Aggregation and Logging”):

```
// scoped to S1a/S1b: Synchronization / Aggregation / Logging
MATCH (caller)-[:CppCalls]->(callee)
WHERE (caller.name CONTAINS '/src/Synchronization/'
      OR caller.name CONTAINS '/include/cpsCore/Synchronization/')
      AND (callee.name CONTAINS '/Aggregation/' OR callee.name CONTAINS '/Logging/')
RETURN DISTINCT
  caller.name AS callerName, caller.symbol AS callerSymbol,
  callee.name AS calleeName, callee.symbol AS calleeSymbol
```

Before this query runs, the running example edge `SynchronizedRunner.runSynchronized→Aggregator.getAll` exists in the graph only as a raw `CppCalls` relationship among 67,104 undifferentiated nodes, with a near-identical sibling edge to `CPSLogger.flush` fired from the same call site; this query is what carves the `S1a/S1b`-scoped subgraph out of the rest of the graph.

5.2.4 Dependency Table Extraction

The `get-interface-dependency-table` skill queries Neo4j for all 11 relationship types and their permitted node-label pairs, producing a compact schema summary used to ground the `text-to-cypher` skill so that generated Cypher queries reference only valid edge types. The resulting architectural dependency diagram was also presented to the expert panel in the H3 evaluation (section 6.4).

5.3 Phase 2 — Guided Instrumentation

5.3.1 Probe Insertion via `csp_matcher`

The `add-runtime-instrumentation` skill drives `csp_matcher` (section 4.4) to wrap the cross-component call sites identified above with a comma-operator trace emission, using the find/replace rules in `add_tracing.txt`: three call-site patterns (`getAll`, `flush`, `convert`), one per distinct cross-component call type the Cypher query surfaced, plus the `CPSLOG` macro patch covering every `RAIILogStream::stream` call site. `apply_tracing.ps1` applies these across all compilation units in `compile_commands.json`:

```
// Example wrapped call site in AggregatableRunner.cpp
(cspTrace("Synchronization", "Aggregation", "Aggregator.getAll"),
agg->getAll<IAggregatableObject>())
```

Each probe emits a structured, pipe-delimited [TRACE] line to stderr when the call fires, capturing wall-clock timestamp, caller/callee component, and function names (full rules in section A.2). Because probes wrap call sites rather than function entries, the trace records that a specific cross-component call executed, not merely that a function was entered.

5.3.2 Trace Format and Output

EventTimestamp	EventName	ClientComponent	ClientFunction	ServerComponent	ServerFunction	RelationshipType	InterfaceNames
----------------	-----------	-----------------	----------------	-----------------	----------------	------------------	----------------

Running the CPSCore test suite under `csp_matcher` instrumentation produced 27 real trace events covering 26 unique client-to-server interaction pairs, genuine runtime captures, not a static projection. Example events:

```
2026-06-18T10:07:15.686645Z | Aggregator.getAll |
Synchronization | AggregatableRunner.notifyAggregationOnUpdate |
Aggregation | Aggregator.getAll | Command | Aggregator

2026-06-18T10:07:15.694645Z | CPSLogger.flush |
Synchronization | SynchronizedRunner.runSynchronized |
Logging | CPSLogger.flush | Command | CPSLogger
```

The full test run passed all 1,425 assertions (section 5.1), confirming instrumentation did not alter runtime behaviour.

5.4 Phase 3 — Neo4j-Based Reachability Filter

Phase 3 applies the Neo4j-based reachability filter (section 4.6.2) to each condition's trace input: C5 filters `trace_slice.txt`, C6 filters `full_trace.txt`, C4 reads `trace_slice.txt` without the filter, and C7–C9 pass C2, C3, and C4 candidate lists to a two-prompt Large Language Model (LLM) chain.

For CPSCore, `trace_slice.txt` contains 8 sequence events and both ground-truth interaction edges for S1a, and 5 sequence events and 3 interaction edges for S1b. Setup-phase events (`Aggregator.add` firing before either runner starts) are present in both traces and are removed by the C5 filter; S2 and S3 contain no setup-phase noise.

5.5 Phase 4 — SysML v2 Generation

For S1a, S1b, S2 and S3, the `draw-sysml-sequence-model` and `draw-sysml-structural-model` skills read the `sequence.json` and `interactions.json` files written by condition C5 (guided+reachability) and render each scenario as a SysML v2 interaction

definition, following the schemas in section 4.8. The broader, whole-codebase set of 14 auto-generated scenarios described below is a separate, bulk visualisation: it is generated directly from the unscoped Phase 1 CSV tables, bypassing scenario scoping and synthesis entirely, and serves to illustrate coverage across CPSCore rather than to feed the H1/H2 evaluation. The generated package declares shared stereotypes (`SynchronousCall`, `ReturnMessage`, `AsynchronousMessage`) and one `part def` per scenario. An excerpt from the generated `TestExecution` scenario:

```
package CPSCoreInteractionModels {
  private import ScalarValues::String;

  part def SynchronousCall :> Message { ... }

  part def TestExecutionScenario :> InteractionScenario {

    part filterTest : Lifeline;
    part catch2      : Lifeline;
    part logger       : Lifeline;
    part enumMap      : Lifeline;

    part m1 : SynchronousCall {
      ref from : Lifeline = filterTest;
      ref to   : Lifeline = catch2;
      attribute :>> label = "1. complete";
    }
    part m6 : SynchronousCall {
      ref from : Lifeline = loggerTest;
      ref to   : Lifeline = logger;
      attribute :>> label = "6. instance";
    }
    ...
  }
}
```

A separate `RedisConnectionScenario` covers the Redis subscriber/publisher initialisation interactions (`connect`, `auth`, `subscribe`, `onConnectionEvent`), spanning the `RedisPublisher`, `RedisSubscriber`, `CppRedisClient`, and `Logger` lifelines.

The complete set of generated SysML v2 sequence and structural diagrams is reproduced in section A.8, covering all 14 auto-generated scenarios (Aggregation, Synchronization, IDC, IPC, Redis, Serial, Signal Handler, Utilities, Data Presentation, and Test Execution flows).

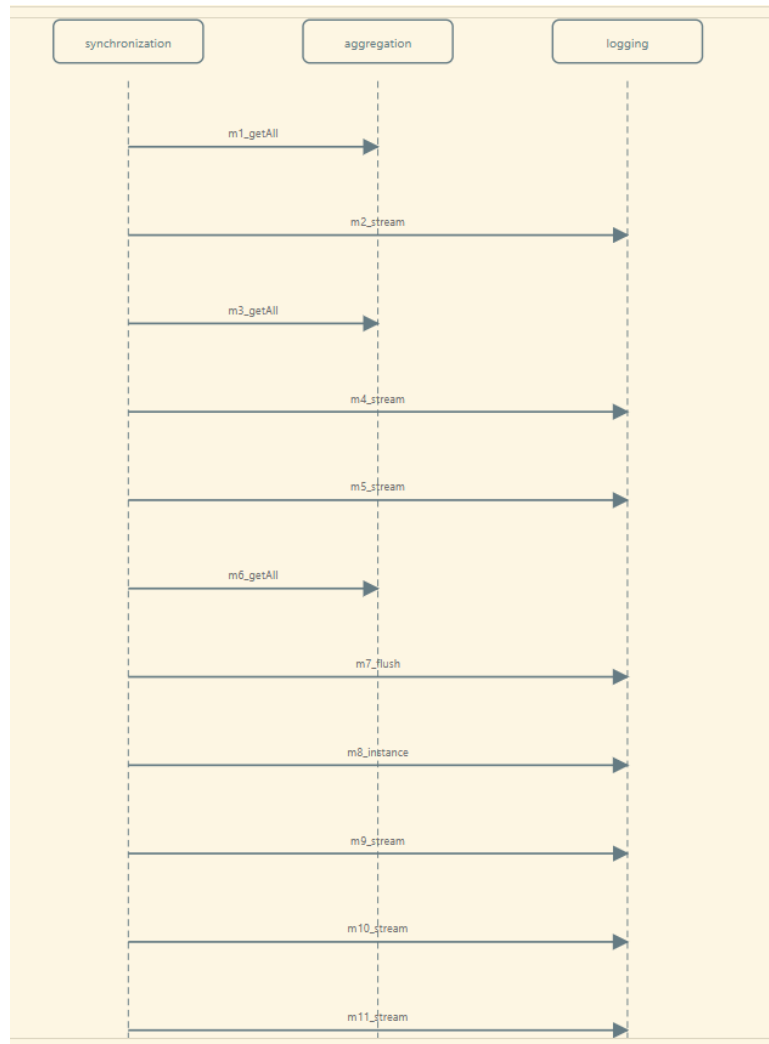


Figure 5.1: An example of the pipeline’s final rendered output: the generated SysML v2 sequence diagram for the whole-codebase *Aggregation* scenario, one of the 14 bulk-generated, unscoped diagrams described above. It is illustrative of output style and coverage only, not one of the scenario-scoped, per-condition reconstructions scored in section 6.2 (those are reproduced separately in section A.4). The complete set of bulk-generated diagrams is reproduced in section A.8.

5.6 Scenario Suite and Experimental Conditions

The stages above produce the raw artefacts (the static sequence dependency table and the real `csp_matcher` trace log) feeding the H1 and H2 experiments in chapter 6. This section documents how those artefacts were assembled into experimental configurations.

5.6.1 Scenario Suite (S1a, S1b, S2, S3)

Four scenarios were defined on CPSCore, all found-in-the-wild: each corresponds to an existing Catch2 test case exercising a real cross-component interaction, none constructed for this evaluation. Two of them, S1a and S1b, exercise the same underlying call chain under different test cases — the happy path and the timeout branch of `SynchronizedRunnerTest`, respectively — covering both an over-approximation blind spot in static analysis (S1a: C2 reports extraneous static edges the trace never fires) and a runtime-suppression blind spot in dynamic analysis (S1b: logging is suppressed by `setLogLevel(NONE)`, so the timeout-warning interaction is visible only to static analysis).

S1a: Aggregator notification chain, happy path. *Primary:* Synchronization. *Targets:* Aggregation, Logging. The observer/notification pattern exercised by `SynchronizedRunnerTest`: `SynchronizedRunner.runSynchronized → Aggregator.getAll`, followed by a logging call. The `boost::signals2` callback chain is invisible in a static include graph, making this a test of IDC topology recovery. *Reference interactions:* 2 (`getAll`, `flush`).

S2: Configuration property mapping. *Primary:* Configuration. *Target:* Logging. A mostly-static, linear chain: `PropertyMapper.add → RAIILogStream.stream`. The easy-case baseline, expected to be reconstructed well even by static-only conditions. *Reference interactions:* 1 (`stream`).

S3: Multi-component runner orchestration. *Primary:* Synchronization. *Targets:* Aggregation, Utilities, Logging. Fan-out from `AggregatableRunner.runAllStages/notifyAggregationOnUpdate` to multiple independent subsystems at once, driven through `MultiThreadingScheduler.schedule/run/runSchedule/stop` and `ObjectHandleContainer.setFromAggregationIfNotSet`. The hardest case for static analysis: one of the three genuine cross-component calls (`Utilities → Logging:stream`) is dispatched through a `std::function`-wrapped thread with no resolvable static call site, so C2 misses it entirely — a recall gap, not a false positive (C2’s precision on S3 is 1.000; it reports zero extraneous edges, only an incomplete set). Intersection (C5) is therefore bounded by C2’s recall on this scenario alone (section 6.2.3). *Reference interactions:* 3 (`Aggregation → Logging:stream`, `Synchronization → Aggregation:getAll`, `Utilities → Logging:stream`); full listing in section A.3.

S1b: Aggregator notification chain, timeout path. *Primary:* Synchronization. *Targets:* Aggregation, Logging. The same call chain as S1a, exercised instead by the timeout branch of `SynchronizedRunnerTest`. The test suppresses all logging via `CPSLogger::instance() -> setLogLevel(LogLevel::NONE)` before triggering the timeout, so the timeout-warning interaction (`SynchronizedRunnerMaster.runStage → RAIILogStream.stream`) is visible in C2’s static call graph but absent from the unfiltered runtime trace (C3)

— a deliberate dynamic blind spot mirroring S1a’s static over-approximation blind spot. *Reference interactions*: 3 (getAll, flush, stream). As with S1a, `CPSLogger::instance()` is excluded from ground truth as a singleton accessor call rather than a genuine interaction (section 5.2.4), and the `Aggregator.add→RAIILogStream.stream` call that fires during test-fixture setup, before either runner starts, is excluded as out of the scenario’s logical scope (section A.3).

Each scenario’s ground truth was first drawn by hand from direct source inspection: the relevant directories, classes and call sites, together with the test case that exercises them, and only then transcribed into the SysML v2 textual reference diagrams used for scoring, keeping the reference independent of the pipeline’s own output. Both forms, for all four scenarios, are reproduced in section A.3.

5.6.2 Nine-Condition Design (C1–C9)

Each scenario was reconstructed independently under nine conditions:

- C1: LLM-only.** Reconstruction by an LLM reasoning over raw source code alone, with no access to the static graph or any trace, a baseline for what an agent can infer without the pipeline. As an Azure-OpenAI-backed condition it is not deterministically reproducible run-to-run, unlike C2–C6.
- C2: Static-only.** Forward `CppCalls*0..8` traversal from scenario entry points in the Neo4j call graph, collecting cross-component edges in traversal order.
- C3: Unfiltered dynamic trace.** All events from `full_trace.txt` with no component-scope or reachability filter applied. Retains intra-component calls and setup-phase events, making this the noisiest dynamic baseline.
- C4: Guided-only.** Events from `trace_slice.txt` (instrumentation probes placed at Neo4j-identified architectural call sites only) with no further reachability filter. Setup-phase events can still fire at instrumented sites, so C4 retains some setup-phase false positives; it is otherwise clean.
- C5: Guided + reachability.** `trace_slice.txt` filtered by a Neo4j `CppCalls*0..8` reachability query scoped to `SCENARIO_ENTRY` functions. Eliminates setup-phase false positives by keeping only events whose caller is reachable from the scenario entry point. Fully deterministic and reproducible.
- C6: Full trace + reachability.** `full_trace.txt` with the same reachability filter as C5. Ablation of C5 that tests whether the filter alone is sufficient regardless of instrumentation strategy.
- C7: LLM + C2 candidates.** Two-prompt LLM chain seeded with C2’s static candidates: the first prompt filters static over-approximations and adds dynamically-dispatched calls; the second orders the sequence. Requires C2 output and Azure credentials.
- C8: LLM + C3 candidates.** Same chain seeded with C3’s unfiltered trace: the first prompt removes intra-component and setup-phase events and adds missed calls.
- C9: LLM + C4 candidates.** Same chain seeded with C4’s guided-trace candidates: the first prompt removes any residual setup-phase false positives and adds missed calls.

C1–C9 test H1 (which evidence source, or combination of sources, most accurately recovers interaction sets and execution sequences). C4 and C5 isolate the contribution of the reachability filter (C5 vs. C4); C5 and C6 isolate the contribution of guided instrumentation (C5 vs. C6). C7, C8, and C9 assess whether LLM post-processing adds value when a prior is already available. H2 is tested separately, by the guided-versus-full instrumentation comparison below (section 5.6.3), not by any pair of the nine conditions above.

5.6.3 Guided versus Full Instrumentation

H2 requires a second axis, orthogonal to the nine conditions above: how the trace itself is collected.

Full instrumentation. `csp_matcher` probes are inserted at every candidate call site in the static graph, independent of scenario.

Code-graph-guided instrumentation. Probes are inserted only at call sites within the scenario scope (section 2.5), computed *before* instrumentation rather than filtered after.

This is what makes the H2 comparison meaningful: whether using the code graph to decide where to look produces a cleaner trace than instrumenting everything and filtering afterwards, at equivalent or better coverage. Results (node/edge/noise reduction, coverage) are reported in section 6.3.2.

Table 5.1: Quantitative outcomes per pipeline stage on CPSCore.

Artefact	Count
Graph node labels	11
Graph relationship types	11
Sequence dependency table rows	2,574
Unique client→server function pairs (static)	2,169
Distinct components in sequence table	121
Instrumentation targets (functions)	9
Source files instrumented	5
Total trace events (<code>csp_matcher</code>)	27
Unique client→server pairs (trace)	26
SysML v2 scenarios generated	14
Test assertions passing post-instrumentation	1,425 / 1,425

Evaluation

This chapter evaluates the GDI pipeline against the three hypotheses (H₁, H₂, H₃) that operationalise the research questions from chapter 1. It reports the design and results of each experiment on the CPSCore case study, a complementary practitioner session on a second industrial codebase and a discussion of threats to validity, including several methodological risks caught during the experiments themselves.

6.1 Evaluation Design

- **H₁** (section 6.2) compares reconstruction accuracy across conditions C₁–C₉ (section 5.6.2) on four scenarios (section 5.6.1), all occurring naturally in CPSCore (S_{1a}, S_{1b}, S₂, S₃).
- **H₂** (section 6.3) compares guided against full instrumentation on structural compression, noise and coverage, with a blinded expert rating of the resulting diagrams.
- **H₃** (section 6.4) tests whether the reconstructed models improve an agent’s performance on a scenario-comprehension task, relative to reasoning over raw source code.

Section 6.7 draws the two research questions’ answers together from these experiments. Section 6.5 reports a fourth, exploratory data point: practitioner feedback from a second, much larger industrial codebase, kept separate from H₁–H₃ since it was not run as a controlled or pre-registered comparison.

6.2 H₁: Does Combining Static and Dynamic Evidence Improve Reconstruction Accuracy?

Hypothesis. Guided instrumentation constrained by a Neo4j reachability filter (C₅) achieves higher reconstruction accuracy than any single-source or unconstrained condition; full-trace instrumentation with the same filter (C₆) matches C₅, confirming the filter alone is sufficient regardless of instrumentation strategy.

6.2.1 Metric

For every condition and scenario, the generated SysML v2 sequence diagram is compared against a manually authored reference diagram over *interaction edges* (unique ordered ⟨sender, receiver⟩ pairs):

$$P = \frac{|\text{gen} \cap \text{ref}|}{|\text{gen}|}, \quad R = \frac{|\text{gen} \cap \text{ref}|}{|\text{ref}|}, \quad F_1 = \frac{2PR}{P + R}$$

Reference diagrams were authored before any condition was scored against them, to avoid tuning the reference to a particular output. This formulation follows the standard information-retrieval convention [21], mirrors its recent use for comparing static against dynamically-sampled call graphs [17], and matches the convention used within the sequence-diagram reconstruction literature itself [27].

A second metric scores the same generated output against the same reference for *execution order*, which the edge-set metric above discards entirely (two conditions can share an identical interaction set while disagreeing completely on the order events fired). Let *gen* and *ref* now denote *ordered* event lists, with repetitions preserved, and let $\text{LCS}(\text{gen}, \text{ref})$ be their longest common subsequence – the longest run of events appearing in both lists in the same relative order, though not necessarily contiguously:

$$P = \frac{|\text{LCS}(\text{gen}, \text{ref})|}{|\text{gen}|}, \quad R = \frac{|\text{LCS}(\text{gen}, \text{ref})|}{|\text{ref}|}, \quad F_1 = \frac{2PR}{P + R}$$

Precision here penalises a condition for inserting extra events (e.g. uncorroborated static edges, or setup-phase trace events the scenario-scoped ground truth excludes); recall penalises it for dropping or misordering genuine ones. Both metrics are reported throughout this section: the edge-set metric answers whether a condition identifies the right *interactions*; the LCS metric answers whether it also gets their *order* right.

6.2.2 Results

Table 6.1: H1 per-scenario results (interaction edges): precision / recall / F1 by condition. Scenarios S1a, S1b, S2, S3 all occur naturally in CPSCore (section 5.6.1).

Condition	S1a	S1b	S2	S3
C1: LLM-only	1.00/1.00/1.00	1.00/1.00/1.00	1.00/1.00/1.00	.75/1.00/.86
C2: Static-only	.50/1.00/.67	.75/1.00/.86	1.00/1.00/1.00	1.00/.33/.50
C3: Unfiltered trace	.29/1.00/.44	.38/1.00/.55	.50/1.00/.67	.30/1.00/.46
C4: Guided-only	.67/1.00/.80	.75/1.00/.86	1.00/1.00/1.00	1.00/1.00/ 1.00
C5: Guided+Reach	1.00/1.00/ 1.00	1.00/1.00/ 1.00	1.00/1.00/ 1.00	1.00/1.00/ 1.00
C6: FullTrace+Reach	1.00/1.00/ 1.00	1.00/1.00/ 1.00	1.00/1.00/ 1.00	1.00/1.00/ 1.00
C7: LLM+C2	1.00/1.00/1.00	1.00/1.00/1.00	1.00/1.00/1.00	.50/1.00/.67
C8: LLM+C3	1.00/1.00/1.00	1.00/1.00/1.00	1.00/1.00/1.00	.50/.67/.57
C9: LLM+C4	.67/1.00/.80	1.00/1.00/1.00	1.00/1.00/1.00	.33/.33/.33

F1 aggregation: macro-average across the four scenarios throughout.

Table 6.2: H1 per-scenario results (execution-order LCS metric): precision / recall / F1 by condition, over the same four scenarios.

Condition	S1a	S1b	S2	S3
C1: LLM-only	1.00/1.00/1.00	1.00/1.00/1.00	1.00/1.00/1.00	.71/.77/.74
C2: Static-only	.50/.25/.33	.75/.60/.67	1.00/1.00/1.00	1.00/.08/.14
C3: Unfiltered trace	.21/1.00/.34	.18/1.00/.30	.50/1.00/.67	.25/1.00/.40
C4: Guided-only	.67/1.00/.80	.56/1.00/.72	1.00/1.00/1.00	.77/1.00/.87
C5: Guided+Reach	1.00/1.00/ 1.00	1.00/1.00/ 1.00	1.00/1.00/ 1.00	1.00/1.00/ 1.00
C6: FullTrace+Reach	1.00/1.00/ 1.00	1.00/1.00/ 1.00	1.00/1.00/ 1.00	1.00/1.00/ 1.00
C7: LLM+C2	1.00/1.00/1.00	1.00/1.00/1.00	1.00/1.00/1.00	.56/.77/.64
C8: LLM+C3	1.00/1.00/1.00	1.00/1.00/1.00	1.00/1.00/1.00	.62/.62/.62
C9: LLM+C4	1.00/1.00/1.00	1.00/1.00/1.00	1.00/1.00/1.00	.58/.54/.56

Table 6.3: H1 macro-averaged results across all four scenarios, interaction edges and execution-order LCS metric.

Condition	Interactions			Sequence (LCS)		
	P	R	F1	P	R	F1
C1: LLM-only	0.938	1.000	0.964	0.929	0.942	0.935
C2: Static-only	0.812	0.833	0.756	0.812	0.482	0.536
C3: Unfiltered trace	0.365	1.000	0.530	0.284	1.000	0.428
C4: Guided-only	0.854	1.000	0.914	0.747	1.000	0.846
C5: Guided+Reach	1.000	1.000	1.000	1.000	1.000	1.000
C6: FullTrace+Reach	1.000	1.000	1.000	1.000	1.000	1.000
C7: LLM+C2	0.875	1.000	0.917	0.889	0.942	0.911
C8: LLM+C3	0.875	0.917	0.893	0.904	0.904	0.904
C9: LLM+C4	0.750	0.833	0.783	0.896	0.885	0.890

6.2.3 Discussion: C5 and C6 Achieve Perfect Scores

Two conditions achieve $F1 = 1.000$ on both metrics across all four scenarios: **C5 (guided + reachability)** and **C6 (full-trace + reachability)**. This confirms the central role of the reachability filter, not the instrumentation strategy: replacing guided with full-trace instrumentation while keeping the filter (C6 vs. C5) leaves every score unchanged.

C4’s two false positives on S1a and S1b are setup-phase events (`Aggregator.add → RAIILogStream.stream`) that fire before the scenario entry point; their callers are not in the Phase 1 reachable function set, so the reachability filter drops them (section 4.6.2), eliminating C4’s remaining false positives and yielding perfect scores.

S2 and S3 have no setup-phase noise in `trace_slice.txt`, so C4 already achieves $F1 = 1.000$ there; C5 matches it. The filter adds value exactly where needed and is otherwise a no-op.

6.2.4 Discussion: Guided vs. Unfiltered and Static Limits

C4 (guided-only, 0.914 / 0.846 macro) substantially outperforms C3 (unfiltered trace, 0.530 / 0.428): `trace_slice.txt` instruments only cross-component architectural call sites, so intra-component calls never appear there, while `full_trace.txt` captures all calls including numerous intra-component ones. C3’s false positives are real runtime events; they are simply not scenario-relevant.

C2 (static-only, 0.756 / 0.536 macro) is structurally capped on S3 by an indirect dispatch: `Utilities` → `Logging::stream` is routed through a `std::function`-wrapped thread with no resolvable static call site, so `CppCalls` never records it. C2’s precision on S3 is 1.000 (it reports no extraneous edges, only an incomplete set); its recall is 0.333 (one of three genuine edges). No Cypher refinement can recover a call site that is structurally absent from the call graph. C5 and C6 recover it because the trace captures the executed call.

6.2.5 Discussion: LLM-Augmented Conditions Underperform Their Counterparts

C7 (LLM + C2 candidates) substantially recovers C2’s recall gap, matching C1’s interactions macro-F1 on three of four scenarios. It still fails on S3: the LLM adds static-analysis-derived edges it cannot verify from source alone, introducing two false positives ($F1 = 0.667$ on S3 interactions).

C8 (LLM + C3) partially denoises the raw trace: S3’s complex intra-component noise is the hardest case, leaving residual false positives ($F1 = 0.571$ on S3 interactions vs. C3’s 0.462).

C9 (LLM + C4) actually hurts: macro interactions F1 drops from C4’s 0.914 to 0.783. On S3, the LLM over-filters — it misclassifies a genuine interaction as setup-phase noise and drops it ($F1 = 0.333$ on S3 interactions). This is the general risk when the prior is already precise: the LLM cannot reliably distinguish genuine from spurious edges in a clean input, and errors become counterproductive.

Which condition to prefer in practice: C5 or C6 guarantee perfect reconstruction where Neo4j is available; C4 is the strongest deterministic option without the Neo4j-based reachability filter; C7 is the best LLM-only option when a static prior is available and recall matters more than precision on S3.

6.3 H2: Does Guided Instrumentation Improve Scenario Focus?

Hypothesis. Code-graph-guided instrumentation (section 5.6.3) suppresses irrelevant execution detail relative to full, unguided instrumentation.

H2 has a quantitative arm (structural compression, noise, coverage, and a complementary reconstruction-accuracy check reusing the H1 conditions and metrics) and a qualitative arm (blinded expert rating). All were needed: H2 is fundamentally a claim about human-perceived usability, which the quantitative metrics alone cannot establish.

6.3.1 Metrics

Let G_{full} , G_{guided} denote the property-graph subsets from full and guided instrumentation, and R the scenario-relevant reference node set (shared with H_1):

$$\text{NodeReduction} = 1 - \frac{|V(G_{\text{guided}})|}{|V(G_{\text{full}})|}, \quad \text{EdgeReduction} = 1 - \frac{|E(G_{\text{guided}})|}{|E(G_{\text{full}})|}$$

$$\text{Noise}(G) = \frac{|V(G) \setminus R|}{|V(G)|}, \quad \text{Coverage}(G) = \frac{|V(G) \cap R|}{|R|}$$

Noise and Coverage are the retained-set analogues of $1 - \text{Precision}$ and Recall [21]; the noise concept is adapted from its use in architecture recovery evaluation [5], applied here to instrumentation scoping rather than clustering. A configuration that reduces noise by discarding relevant nodes along with irrelevant ones is not an improvement; coverage rules that failure mode out.

6.3.2 Quantitative Results

Table 6.4: H2 aggregate results: guided vs. full instrumentation (mean across S1a, S1b, S2, S3).

Metric	Full	Guided	Reduction
Nodes (lifelines)	6.0	3.0	−50%
Interaction edges	9.0	2.7	−70%
Coverage	1.000	1.000	0%
Noise	0.704	0.000	−100%

Guided instrumentation reduced retained nodes by 50% and interaction edges by 70%, driving noise to 0 while holding coverage at 1.000: every scenario-relevant interaction survived the reduction. Full-trace false positives are real runtime events from non-primary components active in the same test run (*e.g.* `Aggregation→Logging:stream`), architecturally valid but irrelevant to the scenario at hand.

The quantitative arm above is therefore scoped to S1–S4, where full instrumentation actually produces excess events for guided instrumentation to remove.

6.3.3 Qualitative Arm: Blinded Expert Rating

Structural compression alone cannot establish that guided instrumentation produces a *more usable* diagram. That is a claim about human perception. A blinded A/B protocol tested this directly: six external domain engineers, with no prior knowledge of the codebase, independently rated diagrams from both configurations without being told which was which, using a rating form and a private decode key (held separately) to keep the assignment blind, with the A/B label randomised per scenario. All six returned completed forms scoring S1–S3 on accuracy, readability and practical usefulness, plus free-text comments and an overall preference.

Table 6.5: H2 qualitative results: mean blinded expert ratings (1–5 scale, $n = 6$ raters) for full vs. guided instrumentation, per scenario and rating dimension.

Scenario	Accuracy		Readability		Usefulness	
	Full	Guided	Full	Guided	Full	Guided
S1	2.33	4.67	2.33	4.83	3.17	4.50
S2	2.33	4.17	2.33	4.83	2.67	2.83
S3	2.50	5.00	2.67	5.00	3.00	4.67
Mean	2.39	4.61	2.44	4.89	2.94	4.00

Results Guided instrumentation was never rated below full instrumentation on accuracy or readability by any rater on any scenario (34 of 36 comparisons strictly preferred, 2 exact ties). Across all $6 \times 9 = 54$ rater–scenario–dimension comparisons, guided was rated strictly higher in 47, tied in 6, and lower in exactly 1. Averaged across all scenarios and dimensions, guided scored 4.50 against 2.59 for full instrumentation.

The larger panel did not change the direction of the effect but surfaced heterogeneity a two-rater sample could not. Usefulness is unanimous for guided on S1 and S3 but genuinely split on S2, the one scenario where guided instrumentation collapses seven repeated `Configuration→Logging` property-read calls into a single edge: three raters tied it, one preferred full, two preferred guided. Free-text comments suggest this reflects a real trade-off: collapsing repeated reads discards a *count* some raters consider useful for debugging, independent of accuracy or readability. Full per-rater decomposition and comments are in section A.7.

Taken together, guided instrumentation compresses the graph without losing scenario-relevant nodes (quantitative arm) and is rated more accurate and more readable by every one of six domain experts, on every scenario (qualitative arm). The one qualification is usefulness on S2, where the reduction discards a property-read count several raters found independently useful for debugging.

6.4 H3: Do Reconstructed Models Improve Task Performance?

Hypothesis. Agents equipped with reconstructed structural and sequence diagrams perform scenario-comprehension tasks more accurately than agents relying only on source code and traces.

H3 was scoped to a single downstream task (Scenario Comprehension) to keep the evaluation tractable. A single holistic experiment isolated source code, execution traces, and a rendered diagram individually and in combination, across all four scenarios, S1–S4.

6.4.1 Protocol

Ground truth for this rescoring is the same automatically derived, scenario-scoped interaction set used throughout H1 (`gt_interactions.json` per scenario, section 5.6.1), converted into expected components and expected interactions. Rescoring against the same ground truth H1 uses keeps the two hypotheses consistent with each other. For

each scenario, an agent was prompted to identify components, enumerate interactions and explain the execution flow under one of five conditions:

Source only. Source snippets only.

Trace only. Runtime trace subset only.

Diagram only. A rendered SysML v2 diagram (structural and sequence, `gen_sysml.py`, section 4.8) only.

Source + Trace. Both.

Source + Trace + Diagram. All three.

Component and interaction recall apply the same formulation as H1: a response is credited for a ground-truth interaction only if the source component, target component and bare method name are all genuinely attributed to that specific interaction, not merely present somewhere in the response text. Completeness is scored 1–5 against `scoring/rubric.md` by a single non-blind rater applying the ground truth above.

6.4.2 Holistic Comparison

Table 6.6: H3 holistic comparison: five conditions across all four scenarios, S1–S4.

Condition	Component recall	Interaction recall	Completeness (1–5)
Source only	0.833	0.444	3.00
Trace only	1.000	0.778	4.00
Diagram only	1.000	0.778	4.25
Source + Trace	1.000	1.000	4.75
Source + Trace + Diagram	1.000	1.000	5.00

Diagram-inclusive conditions are never worse than their non-diagram counterparts, and are often better. Diagram only clearly beats source only (0.778 vs. 0.444 interaction recall) and ties trace only, while adding the diagram to source+trace lifts completeness further (5.00 vs. 4.75) at no cost to recall. Source only is the weakest condition throughout, missing a component and interactions on S1 and S4 that trace or diagram evidence make explicit.

The one shared limitation is S3: source only, trace only and diagram only each misattribute one of two `stream` interactions to `Synchronization` rather than their true source component, capping single-evidence interaction recall there at 1 of 3. Only combining source and trace resolves it, by letting the agent cross-reference the trace’s specific caller against the source code’s specific class – which is why source+trace and source+trace+diagram are the only conditions to reach perfect recall.

6.4.3 H3 Conclusion

H3 is supported: diagram-inclusive conditions are never worse than their non-diagram counterparts and clearly outperform source only, the weakest condition on every measure. Diagram only ties trace only on recall but edges ahead on completeness, and adding

the diagram to source+trace raises completeness further without cost to recall. The one qualification is S₃, where a diagram alone – like source or trace alone – cannot resolve which component actually originates a shared interaction name; only combining source and trace does that. section 6.7 revisits RQ₃ in light of this result.

6.5 Practitioner Validation: A Second Industrial Codebase

H₁–H₃ validate the approach in depth on one system (CPSCore) under controlled conditions. This section reports a complementary, exploratory data point. *Renaissance MCP*, the tool built in this project, exposes the Renaissance static-extraction backend and graph-querying approach as an MCP integration inside GitHub Copilot, and was trialled by practitioners at an industrial partner on a second C++ codebase several orders of magnitude larger than CPSCore.

This is not a repeat of H₁/H₂/H₃: it is an unblinded, semi-structured hands-on session and interview study with four engineers, exercising only the static-extraction and graph-querying capability. No runtime instrumentation or dynamic analysis was involved, so it speaks to the static half of the approach only, and is reported because it bears directly on external validity: whether the static-analysis capability holds up on a second, much larger system. Codebase and defect details are kept generic to avoid disclosing partner-internal specifics.

6.5.1 Session Design

Each session had four parts: tool installation, an *estimation game* (guessing then checking quantitative facts about the codebase), a *scenario exercise* investigating a real, previously-resolved engineering issue, and a wrap-up discussion on the tool’s diagram outputs.

Table 6.7: Estimation-game answers, illustrating codebase scale on the second industrial system.

Query	Answer
Method declarations on the largest class (a test helper)	923
Header files not included elsewhere (excl. externals)	9,606
Inclusions of the most-included header file	2,053
Calls to the most-called function	23,846
Events declared by a representative event-heavy interface	118
Call sites of a representative header-declared method	0 (unused)

The scale here (table 6.7) contrasts with CPSCore’s 2,574 static interactions and 121 components (table 5.1).

6.5.2 Scenario Exercise: A Real Engineering Issue

The exercise centred on a previously-resolved defect chosen for matching this thesis’s target problem class: an implicit, event-driven interaction invisible without reconstructing the call/event chain.

Defect summary (generalised, anonymised). A control service exposes interfaces to a client via proxy objects created on connect. On disconnect, most proxies are freed; one was not, due to a missing reset in connection-handling code. After a later restart, the stale proxy silently stopped delivering change-notification events. The defect was visible only by tracing the event path from the reconnect handler, through the stale proxy, to the downstream handler.

Task. Participants used the tool to answer three questions about the already-fixed codebase: which files changed, which test exercises the fix, and where the proxy objects are used. All three are answerable from the static call/containment graph alone, making this a check on the same static capability underlying RQ1 and RQ2, now on an unfamiliar, industrial-scale codebase. Participants correctly identified the modified file, the relevant test, and the proxy’s construction/release sites, the components underlying the actual fix.

6.5.3 Interview Findings

Four practitioners were interviewed: a software designer (P1), a product owner for an embedded control service (P2), a senior architect for inter-component synchronisation (P3), and an algorithm engineer (P4). Interviews covered general usefulness and six diagram types: class structure (class diagram, polymetric view), object interaction (call graph, dependency diagram), and system overview (structural, sequence diagram).

All four use AI coding assistants daily and saw value in architecture/dependency information. P3 reported completing an investigation in a five-minute session that had previously taken roughly five months, a striking self-report, but a single, unverified, retrospective comparison, reported as anecdote rather than measured effect. P2 noted the graph-based tool located a stub implementation whose concrete definition lived outside the visible workspace, a failure mode of code-search-based assistants. P4 found it most useful for onboarding onto unfamiliar code and locating coverage gaps.

Feedback on diagrams converged on a pattern. Dependency and structural diagrams were consistently rated useful, and in P3’s case judged cleaner than the team’s existing modelling-tool output. Sequence diagrams were valued for surfacing unexpected race conditions and cross-component effects (P1, P3, P4), but all three found event/message labels (*e.g.* a generic `Subscribe`) too abstract without inline detail, and requested source class/line number on hover. The class diagram and polymetric view drew the most mixed feedback: three of four preferred the plain class diagram, and the polymetric view’s size-based encoding was repeatedly flagged as measuring the wrong thing (P2, P4 suggested cyclomatic complexity or requirements traceability instead). Call graphs were the most polarising output: redundant with existing build-system views for P4, not what P3 expected, while P2 rated the equivalent dependency diagram positively for indicating “hops” between components.

Three requests recurred across interviews, independent of diagram type: interactivity (static diagrams compared unfavourably to tools allowing search/filter/collapse, P1–P3); non-code context such as timing, deployment configuration and product variant (P2, P4), a direct instance of the construct-validity limitation already noted for this thesis’s own evaluation, since the graph only contains what was extracted; and invocation friction, with some participants needing unusually explicit phrasing to trigger the tool (P1, P4).

6.5.4 Relationship to This Thesis's Contribution

The class diagram and polymetric view are contributed by other GDI members and fall outside this thesis's scope; they are reported only as context. The findings most relevant here are the positive reception of the dependency and structural diagrams (descended from the interface dependency table, section 5.2.4) and the actionable critique of sequence-diagram labelling (relevant to SysML v2 generation, section 4.8). The hover-detail request is a low-effort extension of the existing `sourceLineNumber` property already carried on `CppCalls` edges (section 6.8), noted as future work (section 7.5).

The proxy-leak defect (section 6.5.2) is an interaction architecturally present in source but only exercised under a specific failure condition (reconnect after a missed release). Participants resolved it using only the static graph, the same evidence source as C2, which recovers architecturally-present interactions regardless of whether they were exercised in any particular run (section 6.2.3). This is informal, but convergent, evidence that static extraction alone remains useful for fault-path investigation on a second, much larger codebase.

The same architecture also motivates, without measuring, a prediction about combining on this system (sections 5.1.1 and 6.2.3). S1a and S1b show that the guided+reachability filter (C5) eliminates false positives by scoping the trace to only the functions reachable from the scenario's entry points in the static call graph – not by requiring agreement between two independently-erring sources. Unlike CPSCore, this codebase uses proxy objects, virtual interfaces (118 events on one representative interface) and dynamic connect/disconnect, conditions under which static analysis is more likely to over- or under-approximate on its own – exactly the setting where the static call graph's reachability scope may cut a broader set of irrelevant trace events. P2's stub-resolution observation, where the graph-based tool located a definition invisible to code search, is consistent with static reachability carrying signal here; but the session collected no dynamic evidence, so it cannot confirm whether the guided+reachability filter would achieve perfect F1 on this codebase. That mechanism was measured only on CPSCore, via S1a, S1b, S2 and S3 (section 6.2.3). Testing the guided+reachability pipeline on a system of this scale and dispatch complexity is left to future work (section 7.5).

6.6 Discussion: Cross-Cutting Pattern

A single pattern runs through the H1 results: additional evidence helps when it disagrees *productively* with the primary source, by independently confirming which of each source's edges are genuine, and is otherwise redundant or actively harmful.

- H1.** Guided instrumentation combined with the static-graph reachability filter (C5) achieves perfect $F1 = 1.000$ on all four scenarios. Guided instrumentation alone without the filter (C4) reaches macro $F1 = 0.914$, already exceeding either source alone; the reachability scope closes the remaining gap. The full-trace variant with the same filter (C6) also reaches 1.000, confirming the filter is the decisive factor, not the instrumentation strategy.

- H2.** Guided instrumentation removes noise without losing coverage, and the panel confirms this translates into perceived usability – except on S2, where the removed information was itself useful for debugging.
- H3.** Diagram-inclusive conditions beat source alone and are never worse than their non-diagram counterparts, but the diagram alone still shares source and trace’s S3 attribution error; only combining source and trace resolves it, by letting the agent cross-reference the trace’s specific caller against the source code’s specific class (section 6.4.2).
- Practitioner session.** The proxy-leak defect was resolved using only the static graph; with no dynamic evidence collected, this speaks to static extraction’s standalone value, not to the guided+reachability mechanism itself (section 6.5).

The pipeline’s precision benefit is therefore conditional on the static call graph providing an accurate reachability scope for the scenario’s entry points – useful for engineers deciding when the guided+reachability strategy is appropriate. H3 adds a downstream caveat: a reconstructed diagram summarises what the pipeline concluded, but attributing an interaction to its specific source component still calls for cross-referencing source and trace directly.

This generates a testable prediction: on a codebase with more complex dispatch (proxy objects, virtual interfaces, unlike CPSCore), the static call graph’s reachability scope may identify a broader set of irrelevant trace events, giving the filter more false positives to suppress while still preserving the scenario’s genuine edges. The practitioner codebase looks like such a system; testing the guided+reachability pipeline there remains future work (section 7.5).

6.7 Answering the Research Questions

RQ1: Hybrid vs. Individual Sources

Does combining a static call graph with runtime execution traces reconstruct scenario-specific Interaction Models more accurately than either source alone?

Combining improves reconstruction to perfect accuracy across all four scenarios. Condition C5 (guided instrumentation + static reachability filter) achieves $F_1 = 1.000$ on every scenario (S1a, S1b, S2, S3), against 0.756 for static alone (C2), 0.530 for unfiltered dynamic alone (C3), and 0.914 for guided dynamic without the filter (C4, section 6.2.3). Condition C6 (full-trace + same reachability filter) also reaches 1.000, confirming the reachability scope is sufficient regardless of instrumentation strategy. S1a and S1b – found-in-the-wild instances of the same test exercising its happy path and timeout branch respectively – show the mechanism holds without special construction. Whether this generalises beyond CPSCore depends on whether the static call graph accurately captures scenario-relevant reachability, which the practitioner session (section 6.5) motivates but does not confirm.

RQ2: Guided Instrumentation

Does guiding trace collection with the static call graph remove reconstruction noise without losing scenario coverage?

Applying the scenario scope *before* instrumentation reduces retained nodes by 50% and edges by 70% while holding coverage at 1.000. The blinded expert rating confirms this translates into perceived usability, with one qualification: collapsing repeated reads on S2 discards a count several raters found useful for debugging (section 6.3.3).

RQ3: Sufficiency for Comprehension

How well does a reconstructed SysML v2 Interaction Model consolidate static and dynamic evidence into a single artefact sufficient for system-behaviour comprehension?

H3 (section 6.4) supports the diagram's value but not its strict sufficiency: diagram-inclusive conditions beat source only and are never worse than their non-diagram counterparts, yet the diagram alone still ties trace only and shares its S3 attribution error (section 6.4.2). Combining source and trace resolves that case; adding the diagram on top raises completeness further at no cost to recall. A reconstructed diagram is therefore a useful complement to the underlying evidence, not a sufficient replacement for it.

6.8 Engineering Utility

Beyond precision/recall, a tool development thesis's primary claim is that the tool is useful to engineers. Three utility properties were observed during development.

Pipeline automation. The full path from C* source to a SysML v2 sequence diagram runs by chaining twelve agent skills (table 4.3), all following a fixed rule set (sections 4.1 and 4.6.2). Every skill produces an auditable intermediate artefact (a CSV, the final .sysml file, or interactions.json/sequence.json for the reachability filter step) rather than acting invisibly, and works from the live schema rather than hardcoded labels, so an engineer can inspect, verify or override any step without rerunning extraction (section 4.3).

Scale. A single Cypher query extracted 2,574 ordered interactions from CPSCore – several working days of manual effort – and identified the nine caller functions across five files to instrument; csp_matcher then wrapped those call sites (three find/replace rules plus a logging-macro patch) and captured the trace, with no prior knowledge of the codebase required.

Correctness preservation. Inserting csp_matcher probes did not alter CPSCore's observable behaviour: all 1,425 test assertions passed after instrumentation, and the tool's reversibility property (section 4.4) restores the source tree to a byte-identical pre-instrumentation state.

6.9 Threats to Validity

6.9.1 Internal Validity

Reference diagram subjectivity. Reference diagrams reflect a single engineer’s understanding per scenario; a different reviewer might choose different granularity or lifeline selection.

Instrumentation coverage. Targets were selected by inspecting the static graph, not coverage analysis; a different entry-point selection would likely change the trace and H₁/H₂ result.

6.9.2 External Validity

Single controlled system for H₁/H₂/H₃. CPSCore is a moderate-scale, cleanly modular framework. The practitioner session offers encouraging but unblinded, four-participant qualitative evidence of transfer, not a repeat of the controlled comparisons; applying the full hypothesis tests to a second system remains the strongest way to strengthen external validity.

No scenario exercises the filter at its limit. All four scenarios are found-in-the-wild, strengthening confidence that the guided+reachability mechanism (section 6.2.3) generalises beyond constructed cases. This also means no scenario exercises a path where the reachability filter removes a genuine edge that C₄ would have retained; every C₄ false positive is setup-phase boilerplate absent from the scenario’s architectural scope. Whether the filter can suppress false positives on a scenario with genuine architectural paths outside the entry-point traversal remains untested and is left to future work (section 7.5).

H₃’s interaction scorer does not verify attribution. The automated check credits a ground-truth item if its source, target and method name appear anywhere in a response, without verifying they describe the same interaction – a gap that mattered on S₃, where two ground-truth interactions share the bare name `stream` with different sources. Figures reported here were corrected by manually reading each response (section 6.4.2); a stricter, structurally-aware matcher is left to future work.

6.9.3 Construct Validity

Edge-level evaluation metric. The primary H₁ metric measures precision/recall over unique (sender, receiver) pairs, abstracting over message count and ordering; a complementary LCS-based execution-order metric (section 6.2) addresses ordering directly. Neither metric weights message content or repetition count, so a diagram with the right calls in the right order but the wrong volume would still score perfectly on both.

Conclusion

7.1 Summary & Findings

This thesis addressed the reconstruction of scenario-specific Interaction Models from C++ codebases and execution traces in event-driven systems. Static analysis alone tends to produce over-approximate, scenario-agnostic graphs; trace-based analysis alone tends to under-complete them, since runtime instrumentation is rarely exhaustive. No prior approach appeared to address publish-subscribe topology recovery and scenario-scoped call-graph querying together, or to target SysML v2 as the output formalism.

The approach (chapter 4) combines a static Neo4j call graph with a scenario-scoped runtime trace, filters the trace through a Neo4j-based reachability filter (condition C5), and projects the result into SysML v2 interaction diagrams.

The approach was developed and evaluated on *CPSCore*, an open-source event-driven C++ framework, across four found-in-the-wild scenarios drawn from its existing test suite, including two instances of the same underlying scenario (its happy path and its timeout branch).

The three hypotheses tested to answer the research questions were confirmed, with an important qualification on *when* and *how well* each effect holds:

H1 (confirmed). C5 (guided + reachability) and C6 (full-trace + reachability) achieve perfect F1 on both interaction-set and execution-order metrics across all four scenarios (macro-F1 = 1.000 / 1.000). The reachability filter is the key mechanism: C5 and C6 differ only in instrumentation strategy yet score identically, confirming the filter is sufficient regardless of how the trace is collected. **C4 (guided-only, 0.914 / 0.846) is the strongest baseline without the Neo4j-based reachability filter**, substantially outperforming unfiltered tracing (C3, 0.530 / 0.428); the gap is explained entirely by intra-component false positives in C3's input. **C2 (static-only) is structurally capped on S3** by indirect thread dispatch (macro-F1 = 0.756): no static call site exists for the `textttUtilities` `to` `textttLogging` edge, so the trace-based conditions are the only ones that recover

it. **LLM-augmented conditions (C7–C9) performed worse than their non-augmented counterparts:** C7 recovers most of C2’s recall but still fails on S3; C9 degrades C4’s already-precise output (0.783 vs. 0.914), confirming that LLM post-processing is counterproductive when the prior is clean.

H2 (confirmed). Code-graph-guided instrumentation eliminates all measured noise (50% fewer retained nodes, 70% fewer edges) without losing coverage, and a blinded six-rater panel rated the guided output strictly higher in 47 of 54 comparisons, with the one reversal traceable to a specific trade-off: collapsing repeated property-read calls into a single edge.

H3 (supported). Diagram-inclusive conditions were never worse than their non-diagram counterparts on the scenario-comprehension task tested, and clearly outperformed source code alone (the weakest condition, missing a component and interactions that trace or diagram evidence made explicit). The reconstructed diagram alone tied with runtime traces alone on interaction recall (0.778), sharing the same attribution error on one scenario; only combining source code with runtime traces reached perfect recall, and adding a diagram on top raised completeness further without cost to recall. A reconstructed diagram is, on this task, a useful complement to the underlying evidence rather than a strictly sufficient replacement for it on its own.

As an exploratory complement, a hands-on session and interview study with four engineers on a second, much larger industrial codebase (section 6.5) offers encouraging evidence that the static half of the approach holds at industrial scale, though the session exercised only static extraction and graph querying, with no runtime tracing involved. The key scenario there was a proxy-leak defect whose root cause was an implicit event-driven interaction, invisible without reconstructing the call chain, resolved using only the static graph, consistent with static extraction remaining useful for fault-path investigation independent of runtime access. The industrial system’s architecture (proxy-based, virtual interfaces, dynamic service binding) is the kind where static analysis is more likely to carry its own independent errors, which H1 would predict gives the static-graph reachability filter more trace events to suppress (sections 5.1.1 and 6.2.3); since no dynamic evidence was collected there, this remains a motivated prediction rather than a measured result.

Taken together, a single pattern runs through the H1 results (section 6.6): additional evidence helps when it disagrees productively with the primary source, by independently confirming which of its edges are genuine, and is otherwise redundant – and even a successfully combined, high-precision reconstruction is not automatically a sufficient basis for a downstream comprehension task, which H3 shows needs the underlying source and trace evidence directly. This generates a falsifiable prediction for future work: on systems with cross-module virtual dispatch, static analysis should carry independent errors even on happy-path scenarios, giving intersection more independent error to cancel across a wider range of scenarios than the single instance (`std::function dispatch`) exercised here (section 7.5).

7.2 Practical Implications

The pipeline lets an engineer working on an event-driven C++ system obtain a scenario-specific SysML v2 interaction diagram through natural-language queries against the property graph, rather than manually tracing call chains, cross-referencing headers and correlating log output over several working days. Section 6.8 quantifies this on CPSCore (2,574 interactions extracted in a single query, nine caller functions instrumented automatically, without any manual call-graph reading); switching to a new target requires only re-running the static extractor, since the skills query the live schema rather than hardcoded labels. The practitioner session (section 6.5) suggests this transfer cost is low in practice: four engineers on a much larger industrial codebase used the tool productively within a single session, and rated the structural and dependency diagrams consistently useful for architectural comprehension.

7.3 Contributions

Research contributions.

1. **A static property graph and a separate trace pipeline** whose interaction edge sets are compared while keeping track of which evidence each interaction came from: static, dynamic, or both. Keeping the sources separate but comparable is what lets the evaluation report exactly where each edge originated (section 4.2).
2. **A scenario-scoping method** that trims the call graph to the interactions relevant to one scenario, using plain scenario-scoped Cypher queries, without re-running the analysis from scratch (section 4.6).
3. **An evidence-based account of when combining static and dynamic analysis helps**, and when it does not, summarised above for H1–H3 (sections 6.2.2, 6.3.2 and 6.4.2).

Engineering artefacts.

4. A **small, reversible instrumentation tool** (`csp_matcher`) that inserts and removes trace probes automatically, driven by a simple find/replace specification (section 4.4).
5. A **set of twelve natural-language agent skills** that together run the whole pipeline, from source code to SysML v2 diagram, on any codebase without code changes, every one following a fixed rule set given its inputs (sections 4.3 and 4.6.2).
6. A **way to recover publish-subscribe wiring** that ordinary call-graph analysis cannot see, by matching subscription and publication call sites that share a channel identifier (section 4.5).

7.4 Limitations & Threats to Validity

The most significant limitation is scale: all three hypotheses were evaluated on a single open-source system (≈ 13 kLOC) across four scenarios. CPSCore’s absence of cross-module virtual dispatch means its static call graph disagrees with the trace on *which*

edges are genuine in only one scenario (an indirect thread dispatch), which is presumably why intersection's recall shortfall is concentrated there rather than spread across every scenario; in a larger system with virtual dispatch or dynamic plugin loading, where static analysis is more likely to carry its own independent errors on a wider range of scenarios, intersection's precision advantage would have more disagreement to cancel, but its recall shortfall could also be more widespread. The practitioner session on an industrial-scale codebase offers encouraging qualitative evidence of transfer, but does not substitute for a controlled replication.

H3's diagram-inclusive conditions were shown diagrams rendered from the guided-only condition (C4) rather than from the guided+reachability condition (C5) (section 6.4.2). Its interaction-recall figures also depend on a manual correction to the automated scorer, which cannot tell a genuinely-attributed interaction from one whose components merely co-occur in the response text. The H2 six-rater panel preserved effect direction on every dimension, though a larger sample would strengthen confidence in magnitude. Ground-truth diagrams are inherently subjective, and the edge-set metric does not capture message content or count even where the execution-order metric captures ordering.

7.5 Future Work

Controlled evaluation on a second system. The practitioner session (section 6.5) suggests the static approach transfers to larger, industrial codebases, but only qualitatively. A natural first step would be applying the full, pre-registered H1/H2/H3 tests to that codebase or to other event-driven C++ systems (*e.g.* using ROS 2 or MQTT for publish-subscribe) to turn this encouraging signal into a controlled generalisability result, ideally on a system with cross-module virtual dispatch so that static analysis carries independent errors on more than the single scenario type observed here.

Diagram labelling and interactivity. The practitioner interviews (section 6.5.3) independently converged on two requests: richer, less abstract event/message labels on sequence diagrams (*e.g.* inline source class and line number on hover), and interactive exploration (search, filter, collapse/expand) rather than static, one-shot diagrams. Both are natural extensions of the source-location data the pipeline already carries (section 6.8).

Temporal ordering accuracy. The current evaluation abstracts over message ordering. A metric accounting for the relative order of interactions would give a more complete picture.

Incremental updates. As the codebase evolves, the PG should ideally be updatable incrementally rather than rebuilt from scratch. Neo4j's support for live graph updates makes this tractable, though the scenario-scoping invalidation logic would need careful design.

LLM-assisted diagram naming. Function names in the generated diagrams are currently taken verbatim from source. A post-processing step using an LLM could produce more readable message labels.

Integration with Renaissance DB. The GDI project's architecture model database (Renaissance DB) could serve as a richer source of module-level metadata, potentially

reducing the number of unresolved black-box components.

7.6 Reproducibility

The complete source code, experiment data, skill definitions, and scoring scripts used in this thesis are available at:

<https://github.com/prathikanand7/masters-thesis-work.git>
https://ci.tno.nl/gitlab/anand_krishnan.prathik-tno/cpscore.git

Appendix

A.1 Property Graph Schema: Full Cypher Constraints

The following Cypher statements create the uniqueness constraints and indexes used by the pipeline, using the node labels from table 4.1.

```
// Uniqueness constraints
CREATE CONSTRAINT fn_def_unique IF NOT EXISTS
  FOR (f:CppFunctionDefinition)
  REQUIRE f.qualified_name IS UNIQUE;

CREATE CONSTRAINT fn_decl_unique IF NOT EXISTS
  FOR (f:CppFunctionDeclaration)
  REQUIRE f.qualified_name IS UNIQUE;

CREATE CONSTRAINT decl_unique IF NOT EXISTS
  FOR (d:CppDeclaration)
  REQUIRE d.qualified_name IS UNIQUE;

CREATE CONSTRAINT symbol_unique IF NOT EXISTS
  FOR (s:Symbol) REQUIRE s.name IS UNIQUE;

// Indexes for lookup performance
CREATE INDEX fn_source IF NOT EXISTS
  FOR (f:CppFunctionDefinition) ON (f.source_file, f.line);

CREATE INDEX file_path IF NOT EXISTS
  FOR (f:SourceFile) ON (f.path);
```

A.2 csp_matcher Tracing Rules for CPSCore

The rules file (add_tracing.txt) below specifies the three call-site patterns instrumented by csp_matcher (section 4.4). Each find:/replace: pair wraps a target call

expression with a comma-operator trace emission capturing caller context at the call site.

```

find:    $agg.getAll<$T>()
replace: (std::fprintf(stderr,
    "[TRACE] %s, $callerFunc, %s, Aggregator.getAll,
    Aggregation, $callerClass, %s\n",
    cspTraceTimestamp().c_str(),
    cspTraceCallerComponent(__FILE__).c_str(),
    cspTraceLocation(__FILE__, __LINE__).c_str()),
    $agg.getAll<$T>())

find:    CPSLogger::instance()->flush()
replace: (std::fprintf(stderr,
    "[TRACE] %s, $callerFunc, %s, CPSLogger.flush,
    Logging, $callerClass, %s\n",
    cspTraceTimestamp().c_str(),
    cspTraceCallerComponent(__FILE__).c_str(),
    cspTraceLocation(__FILE__, __LINE__).c_str()),
    CPSLogger::instance()->flush())

find:    EnumMap<RunStage>::convert($stage)
replace: (std::fprintf(stderr,
    "[TRACE] %s, $callerFunc, %s, EnumMap.convert,
    Utilities, $callerClass, %s\n",
    cspTraceTimestamp().c_str(),
    cspTraceCallerComponent(__FILE__).c_str(),
    cspTraceLocation(__FILE__, __LINE__).c_str()),
    EnumMap<RunStage>::convert($stage))

```

The corresponding removal rule (`remove_tracing.txt`) strips trace wrappers via a filter function:

```

find:    ($any, $orig)
filter:  is_trace_remove
replace: $orig

```

The filter (`tracing_filters.cpp`) accepts only comma expressions whose left operand contains the `[TRACE]` marker, so only trace-injected expressions are removed and pre-existing comma expressions are untouched:

```

bool is_trace_remove(int n, const char* const* ns,
    const char* const* vs) {
    const char* text = get_hole(n, ns, vs, "any");
    return text != NULL && strstr(text, "[TRACE]") != NULL;
}

```

A.3 Ground-Truth Reference Diagrams (S1, S2, S3, S4)

The reference edge/sequence sets used as ground truth for the H1 evaluation (section 6.2.2) are derived automatically by `derive_gt.py` from three independent sources – static reachability from each scenario’s entry-point functions, the per-test-case `trace_slice.txt` runtime capture, and architectural edge validation against the clang-uml package diagram – scoped to calls reachable from the scenario’s own entry points so that test-fixture

setup and teardown are excluded (section A.4 reproduces the pipeline’s own edge output for comparison). This automatic derivation superseded an earlier, single manually authored S1 reference diagram that merged two distinct test cases (`SynchronizedRunnerTest` and its timeout branch) into one scenario and retained the `CPSLogger::instance()` accessor call that Stage 3’s extraction rules exclude elsewhere (section 5.2.4); S1 is accordingly now scored as two separate instances, S1 and S4. Figures A.1 to A.4 show the hand-drawn diagrams used to validate this derivation against independent, Doxygen-cross-checked source inspection: an annotated view of the relevant source directories, classes and call sites, with the exercising test case, that a reviewer produced by reading CPSCore’s source directly, independent of the pipeline under evaluation. Each diagram is scoped to its own scenario instance, matching the SysML v2 listings below.

S1 — Synchronized Runner Test (Ground Truth)

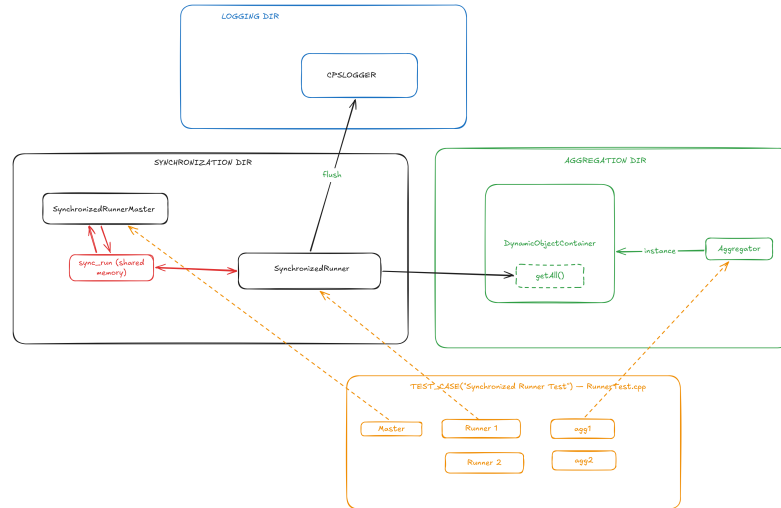


Figure A.1: Source annotation for S1: the Synchronization/Aggregation/Logging call chain exercised by `SynchronizedRunnerTest`’s happy path.

A.3.1 S1: Aggregator Notification Chain, Structural

```
package 'S1-Structural' {
    part def Synchronization {
        part synchronizedRunner : SynchronizedRunner;
    }

    part def Aggregation {
        part aggregator : Aggregator;
    }

    part def Logging {
        part cpsLogger : CPSLogger;
    }
}
```

```

    }

    interface def IAggregator {
        in feature request : 'getAll';
        out feature response : 'ObjectList';
    }

    interface def ICPSLogger {
        in feature flush : 'flush';
    }

    connection : IAggregator
        connect Synchronization::synchronizedRunner to Aggregation::aggregator;
    connection : ICPSLogger
        connect Synchronization::synchronizedRunner to Logging::cpsLogger;
}

```

A.3.2 S1: Aggregator Notification Chain, Sequence

```

package 'S1-Sequence' {

    part sync : Synchronization;
    part agg : Aggregation;
    part log : Logging;

    // SynchronizedRunnerTest: 2x getAll followed by 6x flush
    message m1 from sync.synchronizedRunner to agg.aggregator
        : 'Aggregator.getAll';
    message m2 from sync.synchronizedRunner to agg.aggregator
        : 'Aggregator.getAll';
    message m3 from sync.synchronizedRunner to log.cpsLogger
        : 'CPSLogger.flush';
    message m4 from sync.synchronizedRunner to log.cpsLogger
        : 'CPSLogger.flush';
    message m5 from sync.synchronizedRunner to log.cpsLogger
        : 'CPSLogger.flush';
    message m6 from sync.synchronizedRunner to log.cpsLogger
        : 'CPSLogger.flush';
    message m7 from sync.synchronizedRunner to log.cpsLogger
        : 'CPSLogger.flush';
    message m8 from sync.synchronizedRunner to log.cpsLogger
        : 'CPSLogger.flush';
}

```

A.3.3 S2: Configuration Property Mapping, Structural

```

package 'S2-Structural' {

    part def Configuration {
        part propertyMapper : PropertyMapper;
    }

    part def Logging {
        part railLogStream : RAILLogStream;
    }

    interface def IRAILLogStream {

```

S2 — Configuration Property Mapping (Ground Truth)

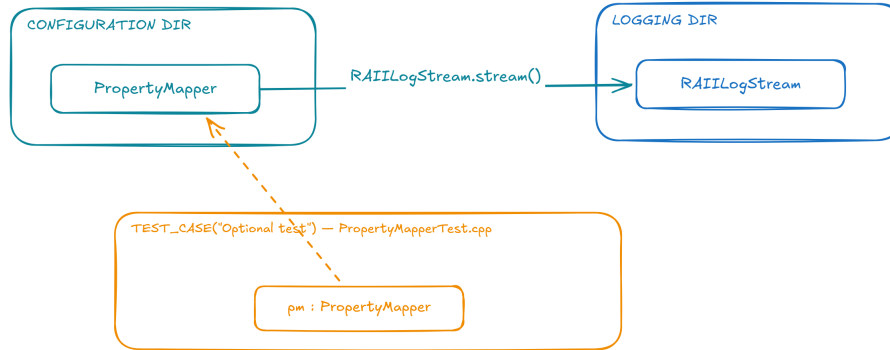


Figure A.2: S2 ground truth, drawn from direct source inspection: the Configuration-to-Logging linear property-mapping chain.

```

    in feature stream : 'stream';
  }

  connection : IRAIILogStream
    connect Configuration::propertyMapper to Logging::raiILogStream;
}

```

A.3.4 S2: Configuration Property Mapping, Sequence

```

package 'S2-Sequence' {

  part cfg : Configuration;
  part log : Logging;

  message m1 from cfg.propertyMapper to log.raiiLogStream
    : 'RAIILogStream.stream'; // PropertyMapper.add
  message m2 from cfg.propertyMapper to log.raiiLogStream
    : 'RAIILogStream.stream'; // PropertyMapper.add (overload)
  message m3 from cfg.propertyMapper to log.raiiLogStream
    : 'RAIILogStream.stream'; // PropertyMapper.addEigen
  message m4 from cfg.propertyMapper to log.raiiLogStream
    : 'RAIILogStream.stream'; // PropertyMapper.addEnum
  message m5 from cfg.propertyMapper to log.raiiLogStream
    : 'RAIILogStream.stream'; // PropertyMapper.addVector
  message m6 from cfg.propertyMapper to log.raiiLogStream
    : 'RAIILogStream.stream'; // PropertyMapper.getChild
  message m7 from cfg.propertyMapper to log.raiiLogStream
    : 'RAIILogStream.stream'; // PropertyMapper.mandatoryCheck
}

```

A.3.5 S3: Multi-Component Runner Orchestration, Structural

```

package 'S3-Structural' {

```

S3 — Multi-Component Runner Orchestration (Ground Truth)

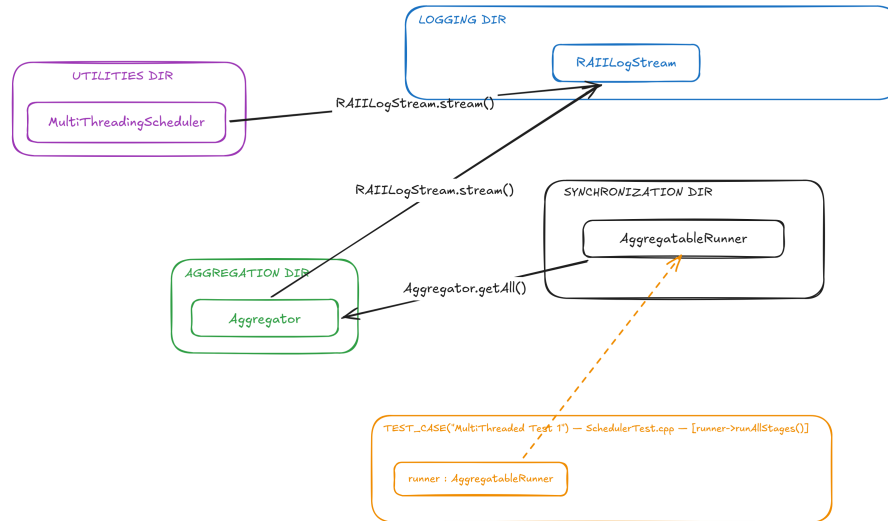


Figure A.3: S3 ground truth, drawn from direct source inspection: the multi-component fan-out from AggregatableRunner into Aggregation, Utilities and Logging (section 5.6.1).

```

part def Synchronization {
    part aggregatableRunner : AggregatableRunner;
}

part def Aggregation {
    part aggregator : Aggregator;
}

part def Utilities {
    part scheduler : MultiThreadingScheduler;
}

part def Logging {
    part raiiLogStream : RAIILogStream;
}

interface def IAggregator {
    in feature request : 'getAll';
    out feature response : 'ObjectList';
}

interface def IRAIILogStream {
    in feature stream : 'stream';
}

connection : IAggregator

```

```

        connect Synchronization::aggregatableRunner to Aggregation::aggregator;
    connection : IRAIILogStream
        connect Aggregation::aggregator to Logging::raiiLogStream;
    // Utilities::scheduler dispatches via a std::function-wrapped thread
    // with no resolvable static call site -- invisible to C2, recovered
    // only by the runtime trace (C3), the S3 shortfall discussed in
    // sec:h1-c4c2.
    connection : IRAIILogStream
        connect Utilities::scheduler to Logging::raiiLogStream;
}

```

A.3.6 S3: Multi-Component Runner Orchestration, Sequence

```

package 'S3-Sequence' {

    part sync : Synchronization;
    part agg  : Aggregation;
    part util : Utilities;
    part log  : Logging;

    message m1 from agg.aggregator to log.raiiLogStream
        : 'RAIILogStream.stream';
    message m2 from sync.aggregatableRunner to agg.aggregator
        : 'Aggregator.getAll';
    message m3 from agg.aggregator to log.raiiLogStream
        : 'RAIILogStream.stream';
    message m4 from sync.aggregatableRunner to agg.aggregator
        : 'Aggregator.getAll';
    // Utilities::scheduler's worker thread logs 9 further times,
    // dynamic-only-visible (see structural connection above)
    message m5 from util.scheduler to log.raiiLogStream
        : 'RAIILogStream.stream';
    message m6 from util.scheduler to log.raiiLogStream
        : 'RAIILogStream.stream';
    message m7 from util.scheduler to log.raiiLogStream
        : 'RAIILogStream.stream';
    message m8 from util.scheduler to log.raiiLogStream
        : 'RAIILogStream.stream';
    message m9 from util.scheduler to log.raiiLogStream
        : 'RAIILogStream.stream';
    message m10 from util.scheduler to log.raiiLogStream
        : 'RAIILogStream.stream';
    message m11 from util.scheduler to log.raiiLogStream
        : 'RAIILogStream.stream';
    message m12 from util.scheduler to log.raiiLogStream
        : 'RAIILogStream.stream';
    message m13 from util.scheduler to log.raiiLogStream
        : 'RAIILogStream.stream';
}

```

A.3.7 S4: Aggregator Notification Chain, Timeout Path, Structural

```

package 'S4-Structural' {

    part def Synchronization {
        part synchronizedRunner : SynchronizedRunner;
    }
}

```

S4 — Synchronized Runner Timeout Test (Ground Truth)

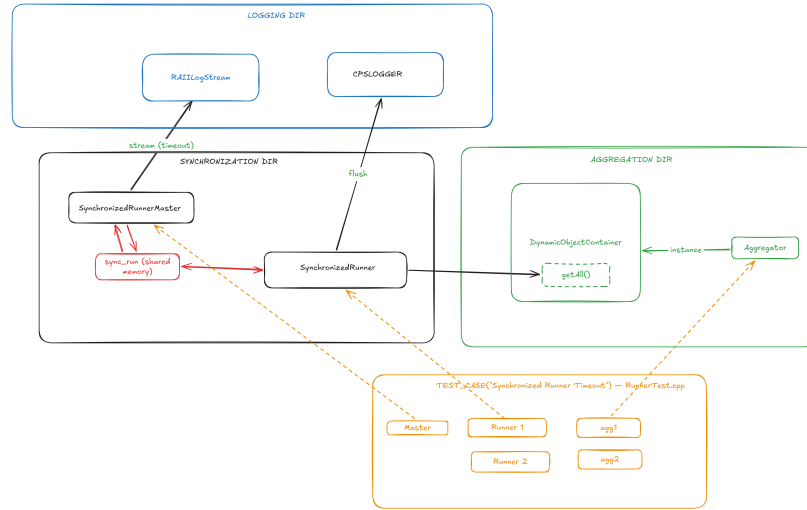


Figure A.4: Source annotation for S4: the same call chain as S1, exercised instead by SynchronizedRunnerTest's timeout branch, which additionally logs via RAIILogStream.stream().

```

part def Aggregation {
  part aggregator : Aggregator;
}

part def Logging {
  part cpsLogger : CPSLogger;
  part raiiLogStream : RAIILogStream;
}

interface def IAggregator {
  in feature request : 'getAll';
  out feature response : 'ObjectList';
}

interface def ICPSLogger {
  in feature flush : 'flush';
}

interface def IRAIILogStream {
  in feature stream : 'stream';
}

connection : IAggregator
  connect Synchronization::synchronizedRunner to Aggregation::aggregator;
connection : ICPSLogger
  connect Synchronization::synchronizedRunner to Logging::cpsLogger;
connection : IRAIILogStream

```

```

    connect Synchronization::synchronizedRunner to Logging::raiiLogStream;
}

```

A.3.8 S4: Aggregator Notification Chain, Timeout Path, Sequence

```

package 'S4-Sequence' {

    part sync : Synchronization;
    part agg  : Aggregation;
    part log  : Logging;

    // SynchronizedRunnerTest, timeout branch: 2x getAll, then flush,
    // the timeout log, and one further flush
    message m1 from sync.synchronizedRunner to agg.aggregator
        : 'Aggregator.getAll';
    message m2 from sync.synchronizedRunner to agg.aggregator
        : 'Aggregator.getAll';
    message m3 from sync.synchronizedRunner to log.cpsLogger
        : 'CPSLogger.flush';
    message m4 from sync.synchronizedRunner to log.raiiLogStream
        : 'RAIILogStream.stream';
    message m5 from sync.synchronizedRunner to log.cpsLogger
        : 'CPSLogger.flush';
}

```

A.4 Generated Pipeline Output

Representative output from the deterministic pipeline (the scenario-scoped intersection of the static and dynamic edge sets, condition C₅, section 6.2.2), shown as component-interaction edge lists (`interactions.json`) extracted from the scenario-scoped edge set.

A.4.1 C₅ (Intersection) Edge Lists by Scenario

Table A.1: Generated component-interaction edges under condition C₅ (intersection, the best-performing evaluated condition on interaction edges, section 6.2.2).

Scenario	Source	Target	Interaction
S1	Synchronization	Aggregation	getAll
S1	Synchronization	Logging	flush
S2	Configuration	Logging	stream
S3	Synchronization	Aggregation	getAll [†]
S4	Synchronization	Aggregation	getAll
S4	Synchronization	Logging	flush
S4	Synchronization	Logging	stream

[†] S₃ is the one scenario where intersection under-recovers: its other two ground-truth edges (`Aggregation→Logging:stream`, `Utilities→Logging:stream`) are

dispatched through a `std::function`-wrapped thread invisible to C2, so intersection collapses to C2's own recall ($P/R/F1 = 1.000/0.333/0.500$, section 6.2.4).

A.4.2 Generated SysML v2 Sequence Model (Excerpt)

The pipeline also produces a SysML v2 sequence model with typed messages. Below is the *Aggregation* scenario, one of 14 auto-generated scenarios in the complete 800-line output:

```
part def AggregationScenario :> InteractionScenario {

    part aggregator : Lifeline;
    part dynamicObjectContainer : Lifeline;
    part logger : Lifeline;
    part aggregatableObject : Lifeline;
    part objectHandleContainer : Lifeline;

    part m1 : SynchronousCall {
        ref from : Lifeline = aggregator;
        ref to   : Lifeline = dynamicObjectContainer;
        attribute :>> label = "1. add";
    }
    part m2 : SynchronousCall {
        ref from : Lifeline = aggregator;
        ref to   : Lifeline = dynamicObjectContainer;
        attribute :>> label = "2. notifyAggregationOnUpdate";
    }
    part m3 : SynchronousCall {
        ref from : Lifeline = aggregator;
        ref to   : Lifeline = dynamicObjectContainer;
        attribute :>> label = "3. clear";
    }
    part m4 : SynchronousCall {
        ref from : Lifeline = aggregator;
        ref to   : Lifeline = dynamicObjectContainer;
        attribute :>> label = "4. empty";
    }
    part m5 : SynchronousCall {
        ref from : Lifeline = aggregator;
        ref to   : Lifeline = dynamicObjectContainer;
        attribute :>> label = "5. size";
    }
    part m6 : SynchronousCall {
        ref from : Lifeline = aggregator;
        ref to   : Lifeline = logger;
        attribute :>> label = "6. stream";
    }
}
```

A.5 Generated Mermaid Diagrams

The pipeline also produces Mermaid diagrams for module-level structural views. The module-dependency diagram below, generated from the static call graph in Neo4j, summarises cross-directory call edges:

```

flowchart TD
    subgraph SRC["src"]
        direction TB
        Aggregation["Aggregation (function defs: 17)"]
        Configuration["Configuration (function defs: 9)"]
        Framework["Framework (function defs: 9)"]
        Logging["Logging (function defs: 12)"]
        Synchronization["Synchronization (function defs: 14)"]
        Utilities["Utilities (function defs: 174)"]
    end

    Utilities -->|"calls (13)"| Logging
    Synchronization -->|"calls (3)"| Aggregation
    Synchronization -->|"calls (3)"| Logging
    Utilities -->|"calls (2)"| Configuration
    Synchronization -->|"calls (2)"| Utilities
    Aggregation -->|"calls (1)"| Utilities
    Logging -->|"calls (1)"| Utilities

```

The runtime-tracing diagram below shows how SignalHandler interacts with logging, IPC, and scheduler components, with static call edges (solid) and runtime subscription edges (dashed):

```

flowchart TD
    subgraph SignalDir["src/Utilities/SignalHandler"]
        sh_exit["subscribeOnExit()"]
        sh_sigint["subscribeOnSigint()"]
        sh_handler["sigIntHandler()"]
    end

    subgraph LoggingAPI["include/cpsCore/Logging"]
        raii_stream["RAIILogStream.stream()"]
    end

    subgraph IPCDir["src/Utilities/IPC"]
        IPC_cpp["IPC.cpp (runtime subscriber)"]
    end

    subgraph SchedulerDir["src/Utilities/Scheduler"]
        Sched_cpp["MultiThreadingScheduler.cpp (runtime subscriber)"]
    end

    sh_exit --> raii_stream
    sh_sigint --> raii_stream
    IPC_cpp -.->|signal subscription| SignalDir
    Sched_cpp -.->|signal subscription| SignalDir

```

A.6 Rendered Diagram Figures

Rendered versions of the diagrams from the preceding sections. Source .mmd files and both PNG and SVG outputs are in the figures/ directory of the repository.

A.6.1 Rendered Diagrams: S1, S2, S3, S4 (Illustrative)

The figures below are rendered artefacts for all four scenarios: the interconnection diagram (the assembled system with real port-to-port connect statements, the same

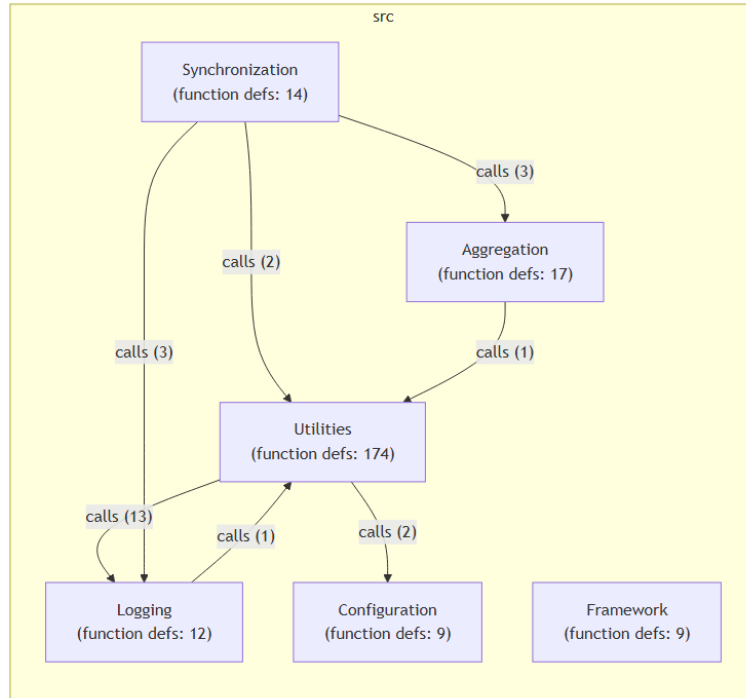


Figure A.5: Module-level dependency structure of CPSCore (cross-directory call edges).

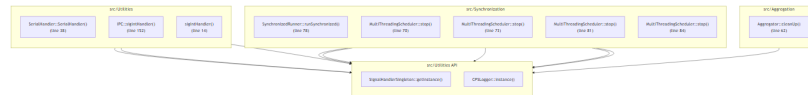


Figure A.6: Cross-module function-call dependencies to singleton APIs.

artefact discussed in section 4.8) and the sequence diagram (draw-sysml-sequence-model). They are rendered from the tagged union (condition C4, section 4.6.2), the pipeline's most permissive combination, not from the higher-precision intersection (C5): so they may not match table A.1's deterministic C5 edge lists exactly (S1's interconnection diagram, for instance, additionally includes the static-only and trace-only edges that C5's intersection filters out, section 6.2.3). Compare against the ground-truth diagrams in section A.3.

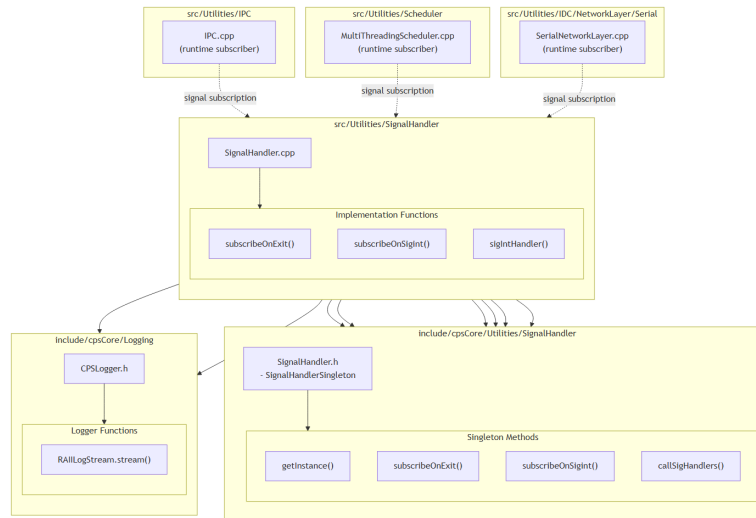


Figure A.7: SignalHandler subsystem: static call edges (solid) and runtime subscriptions (dashed).

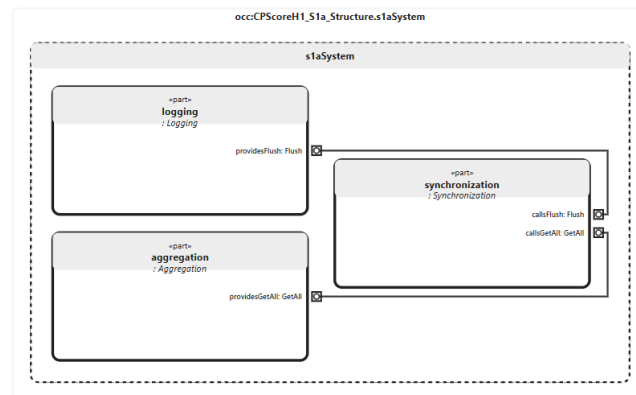


Figure A.8: S₁ (Aggregator notification chain): interconnection diagram.

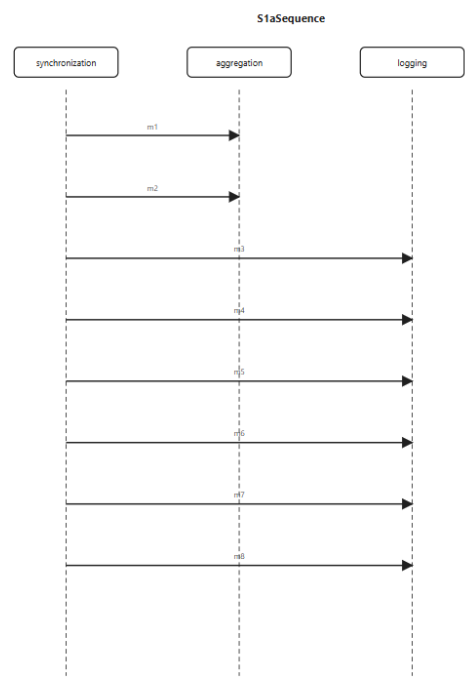


Figure A.9: S1 sequence diagram: message interactions.

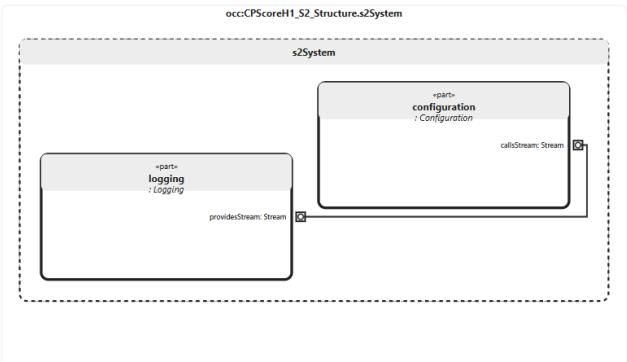


Figure A.10: S2 (Configuration property mapping): interconnection diagram.

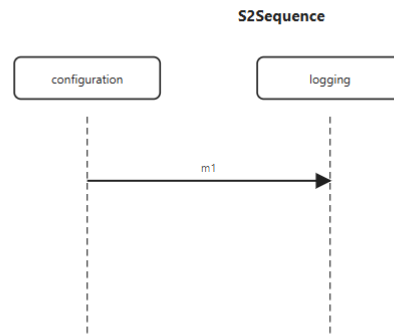


Figure A.11: S2 sequence diagram: Configuration property mapping.

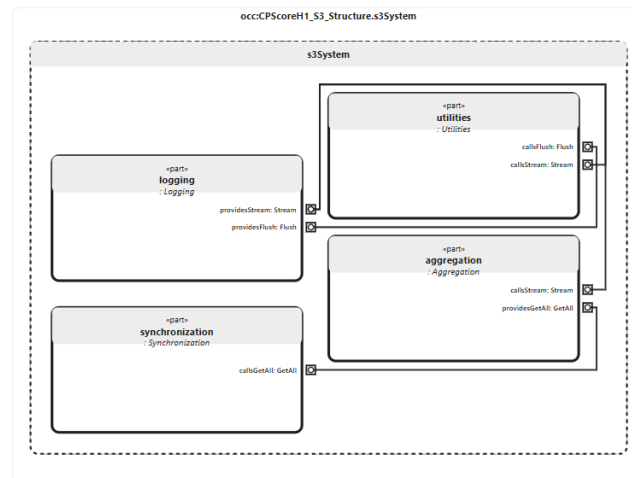


Figure A.12: S3 (multi-component runner orchestration): interconnection diagram.

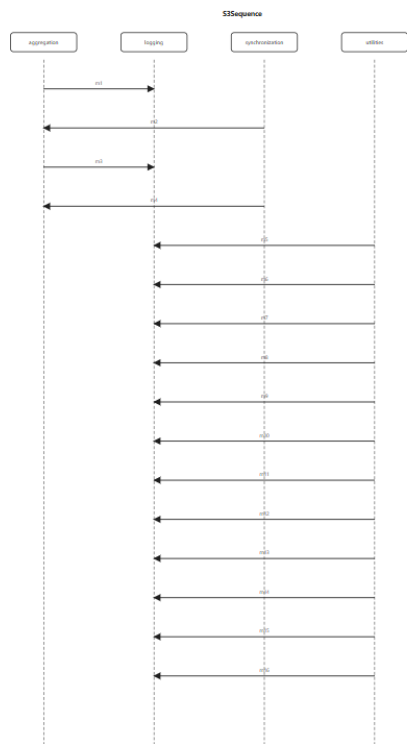


Figure A.13: S3 sequence diagram: multi-component runner orchestration.

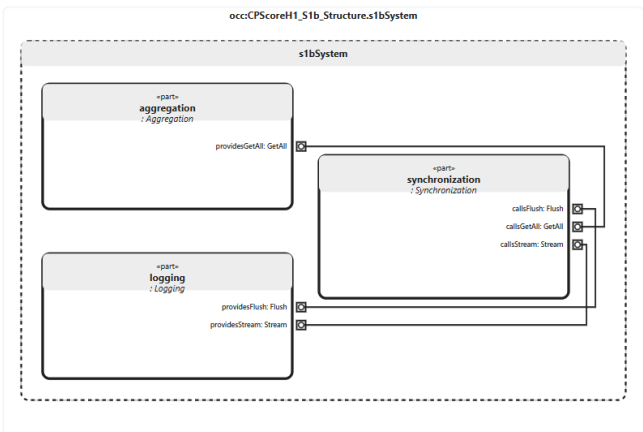


Figure A.14: S4 (Aggregator notification chain, timeout path): interconnection diagram.

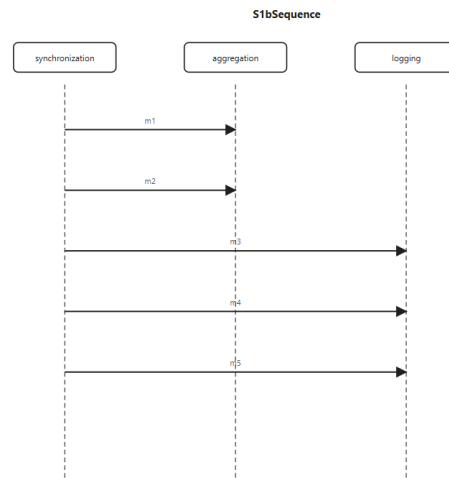


Figure A.15: S4 sequence diagram: Aggregator notification chain, timeout path.

A.7 H2 Qualitative Arm: Per-Rater Decomposition

This appendix records the per-rater decomposition and free-text analysis behind the blinded expert rating study summarised in section 6.3.3. The aggregate results (table 6.5) are given in the main text; the detail below supports the claim that the accuracy/readability effect is unanimous in direction, and that the only genuine split is usefulness on S2.

One rater, R4, consistently rated the two conditions closer together than the other five: R4 accounts for two of the six accuracy ties (S2, S3), one of the six usefulness ties (S1), and the dataset’s only reversal, rating full instrumentation’s usefulness above guided’s on S2 (4 vs. 3). No rater ever rated full instrumentation higher on accuracy or readability on any scenario, so that effect is unanimous in direction (never reversed, though twice tied); usefulness is unanimous on S1 and S3 but genuinely mixed on S2.

The free-text comments explain why. On S1 and S3, five of six raters independently described the guided diagram as retaining only the components named in the scenario context and starting at the correct point in the sequence, while calling the full-instrumentation diagram more complete but cluttered with additional stream events not obviously relevant without prior codebase knowledge. One rater (R1), while still preferring the guided diagram overall, flagged a residual instance call retained in guided S1 as “also superfluous”, a reminder that H2’s noise= 0 result (section 6.3.2) is scoped to the reference set used for scoring, not to what a domain expert reading the diagram could consider extraneous; the two need not coincide.

S2 shows the same pattern in reverse: the guided diagram collapses seven repeated Configuration→Logging property-read calls into a single edge, and five of six raters rated it more accurate and more readable for matching the two components named in the scenario. Usefulness is where the panel genuinely splits. Three raters (R1–R3) tied

it, each independently citing the same reason: collapsing the reads loses a *count* they considered relevant to debugging, even though it played no part in their accuracy or readability judgement. R4 rated full instrumentation *more* useful on S2, for a distinct reason: with the guided diagram alone “it is not clear what triggers a logging event.” Only R5 and R6 rated guided as more useful outright. Usefulness on S2 is therefore a genuine three-way split (tied, reversed, and in favour of guided) rather than the clean 2-rater tie originally reported from the smaller panel.

Several raters converged independently on the same suggestions for handling high-volume `stream` events: R3 and R6 suggested a toggle/filter to distinguish them rather than always including or omitting them; R2 suggested recording the specific property value emitted by each `stream` call, so the collapsed S2 edge stays useful for debugging without losing its readability advantage; and R3 additionally proposed a two-stage workflow: full instrumentation to see where an interaction sits in the broader flow, then guided instrumentation to reason about the scenario itself. This reframes full instrumentation as a complementary, coarser-grained view rather than a failed alternative.

A.8 SysML v2 Generated Diagrams

The following figures show the SysML v2 diagrams generated for the CPSCore case study: structural (component connectivity) and sequence (interaction flow) diagrams for the 14 bulk, whole-codebase scenarios (section 5.5), generated directly from the unscoped static sequence dependency table rather than from any scenario-scoped H1 condition.

A.8.1 Structural Diagrams

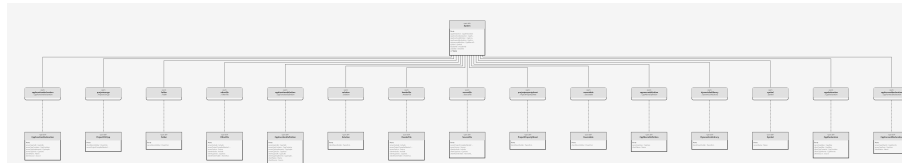


Figure A.16: Complete CPSCore component model: all part definitions and interface connections.

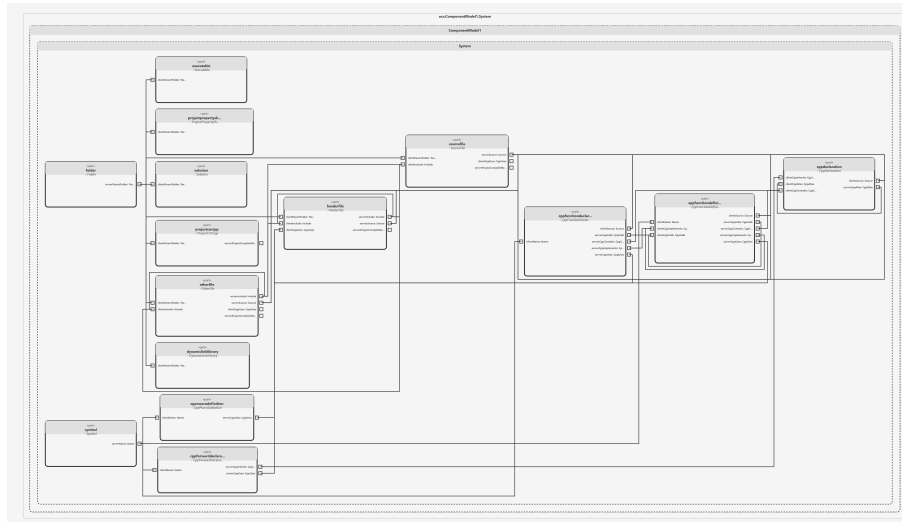
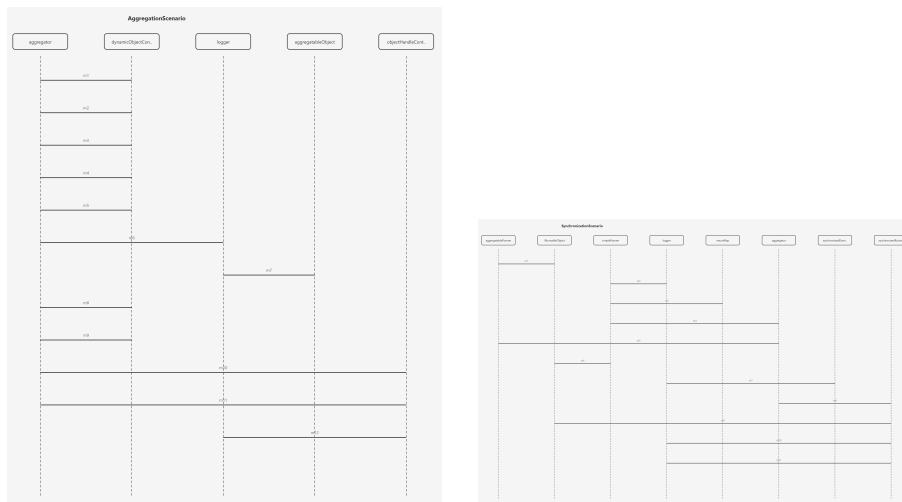


Figure A.17: Node-type hierarchy and property graph schema relationships (13 node labels).

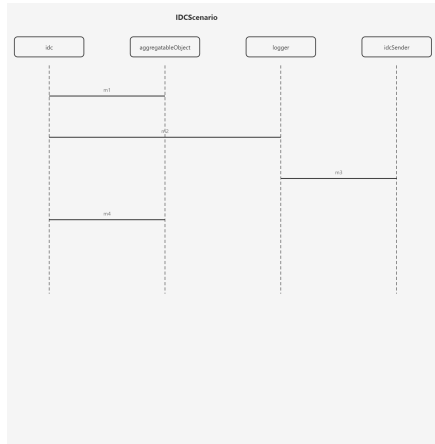


(a) Aggregation scenario: component wiring and lifecycle interactions (Aggregator, DynamicObjectContainer, Logger, AggregatableObject).

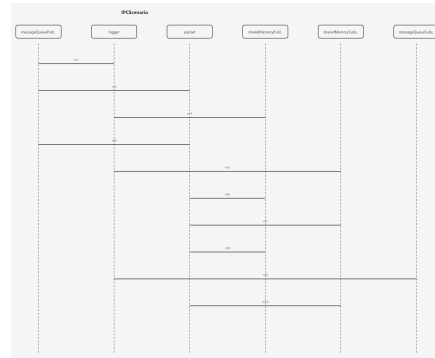
(b) Synchronization scenario: run-stage scheduling interactions (AggregatableRunner, RunnableObject, SimpleRunner, Logger, EnumMap, Aggregator, SynchronizedRunnerMaster, SynchronizedRunner).

Figure A.18: SysML v2 sequence diagrams: Aggregation (left) and Synchronization (right).

A.8.2 Sequence Diagrams

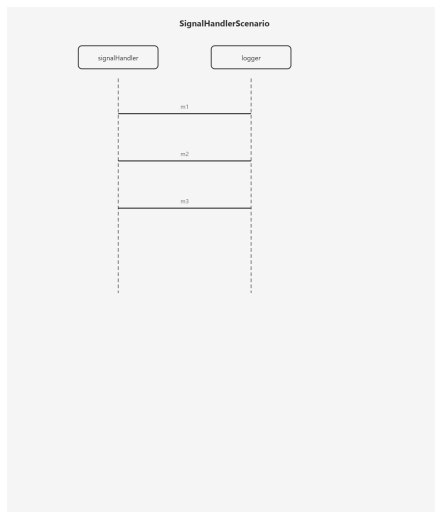


(a) IDC scenario: inter-component communication through the IDC abstraction layer (IDC, AggregatableObject, Logger, IDCSender).

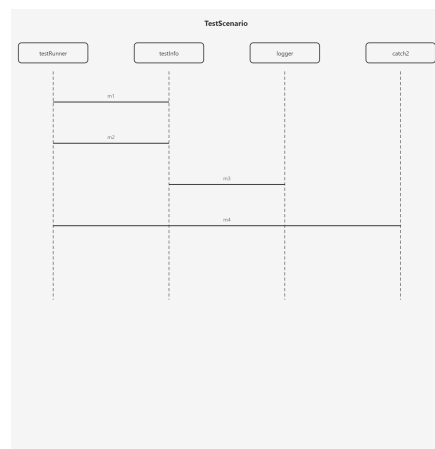


(b) IPC scenario: message queue and shared memory publisher/subscriber interactions (MessageQueuePublisher, Logger, Packet, SharedMemoryPublisher, SharedMemorySubscriber, MessageQueueSubscriber).

Figure A.19: SysML v2 sequence diagrams: IDC (left) and IPC (right).

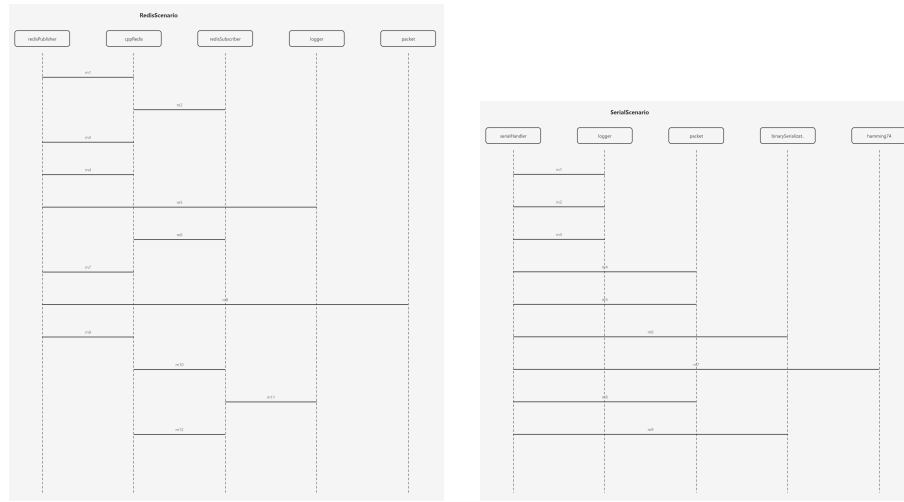


(a) Signal handler scenario: signal handling to logging interaction path (SignalHandler, Logger).



(b) Test scenario: test runner and test info interactions (TestRunner, TestInfo, Logger, Catch2).

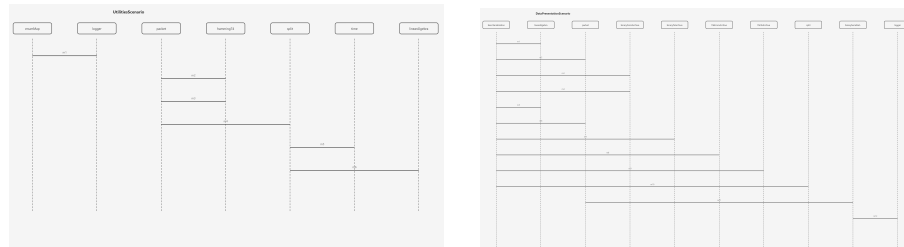
Figure A.21: SysML v2 sequence diagrams: Signal handler (left) and Test (right).



(a) Redis scenario: Redis-based publish-subscribe communication path (RedisPublisher, CppRedis, RedisSubscriber, Logger, Packet).

(b) Serial scenario: data serialisation interactions (SerialHandler, Logger, Packet, BinarySerialization, Hamming74).

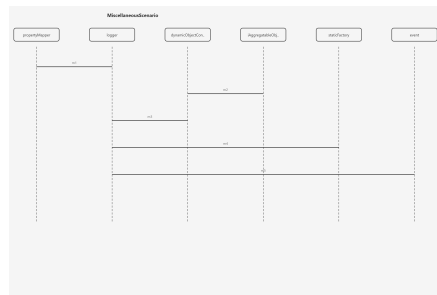
Figure A.20: SysML v2 sequence diagrams: Redis (left) and Serial (right).



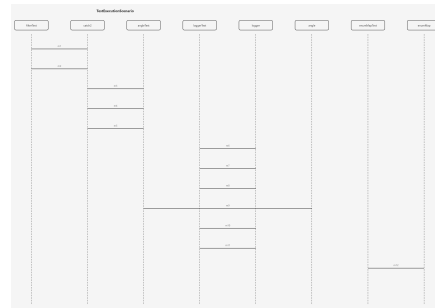
(a) Utilities scenario: cross-utility interactions (EnumMap, Logger, Packet, Hamming74, Split, Time, LinearAlgebra).

(b) Data presentation scenario: Hamming74 encoding/decoding and packet-level interactions.

Figure A.22: SysML v2 sequence diagrams: Utilities (left) and Data presentation (right).



(a) Miscellaneous scenario: PropertyMapper, Logger, DynamicObjectContainer, IAggregatableObject, StaticFactory, and Event interactions.



(b) Test execution scenario: test framework interactions (FilterTest, Catch2, AngleTest, LoggerTest, Logger, Angle, EnumMapTest, EnumMap).

Figure A.23: SysML v2 sequence diagrams: Miscellaneous (left) and Test execution (right).

References

- [1] H. Abdelmalek, I. Khriss and A. Jakimi. “Towards a Hybrid Approach to Reverse Engineer Behavioral UML Diagrams from Source Code”. In: *Journal of Theoretical and Applied Information Technology* 101.22 (2023). No DOI registered for this article as of verification (2026); URL retrieved directly from publisher., pp. 7425–7434. url: <https://www.jatit.org/volumes/Vol101No22/35Vol101No22.pdf>.
- [2] R. Angles, M. Arenas, P. Barceló, A. Hogan, J. Reutter and D. Vrgoc. “Foundations of Modern Query Languages for Graph Databases”. In: *ACM Computing Surveys* 50.5 (2017), 68:1–68:40. doi: 10.1145/3104031.
- [3] C. Baidada, E. M. Bouziane and A. Jakimi. “A New Approach for Recovering High-Level Sequence Diagrams from Object-Oriented Applications Using Petri Nets”. In: *Procedia Computer Science* 148 (2019), pp. 323–332. doi: 10.1016/j.procs.2019.01.040.
- [4] L. C. Briand, Y. Labiche and J. Leduc. “Toward the Reverse Engineering of UML Sequence Diagrams for Distributed Java Software”. In: *IEEE Transactions on Software Engineering* 32.9 (2006), pp. 642–663. doi: 10.1109/TSE.2006.96.
- [5] E. Constantinou, G. Kakarontzas and I. Stamelos. “An Automated Approach for Noise Identification to Assist Software Architecture Recovery Techniques”. In: *Journal of Systems and Software* 107 (2015), pp. 142–157. doi: 10.1016/j.jss.2015.05.065.
- [6] B. Cornelissen, A. Zaidman and A. van Deursen. “A Controlled Experiment for Program Comprehension Through Trace Visualization”. In: *IEEE Transactions on Software Engineering* 37.3 (2011), pp. 341–355. doi: 10.1109/TSE.2010.47.
- [7] B. Cornelissen, A. Zaidman, A. van Deursen, L. Moonen and R. Koschke. “A Systematic Survey of Program Comprehension through Dynamic Analysis”. In: *IEEE Transactions on Software Engineering* 35.5 (2009), pp. 684–702. doi: 10.1109/TSE.2009.28.
- [8] W. De Pauw, D. Lorenz, J. Vlissides and M. Wegman. “Execution Patterns in Object-Oriented Visualization”. In: *Proceedings of the 4th USENIX Conference on Object-Oriented Technologies and Systems (COOTS)*. 1998, pp. 219–234. url: https://www.usenix.org/legacy/publications/library/proceedings/coots98/full_papers/depauw/depauw.pdf.

- [9] S. Ducasse and M. Lanza. “The Class Blueprint: Visually Supporting the Understanding of Classes”. In: *IEEE Transactions on Software Engineering* 31.1 (2005), pp. 75–90. doi: 10.1109/TSE.2005.14.
- [10] A. Egyed. “A Scenario-Driven Approach to Trace Dependency Analysis”. In: *IEEE Transactions on Software Engineering* 29.2 (2003), pp. 116–132. doi: 10.1109/TSE.2003.1178051.
- [11] J. Ferrante, K. J. Ottenstein and J. D. Warren. “The Program Dependence Graph and Its Use in Optimization”. In: *ACM Transactions on Programming Languages and Systems* 9.3 (1987), pp. 319–349. doi: 10.1145/24039.24041.
- [12] M. Gabel and Z. Su. “Javert: Fully Automatic Mining of General Temporal Properties from Dynamic Traces”. In: *Proceedings of the ACM SIGSOFT Symposium on the Foundations of Software Engineering (FSE)*. 2008, pp. 339–349. doi: 10.1145/1453101.1453150.
- [13] H. Grati, H. Sahraoui and P. Poulin. “Extracting Sequence Diagrams from Execution Traces Using Interactive Visualization”. In: *17th Working Conference on Reverse Engineering (WCRE)*. 2010, pp. 87–96. doi: 10.1109/WCRE.2010.18.
- [14] D. Grove, G. DeFouw, J. Dean and C. Chambers. “Call Graph Construction in Object-Oriented Languages”. In: *Proceedings of the 12th ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*. 1997, pp. 108–124. doi: 10.1145/263698.264352.
- [15] A. Hamou-Lhadj and T. C. Lethbridge. “Summarizing the Content of Large Traces to Facilitate the Understanding of the Behaviour of a Software System”. In: *Proceedings of the IEEE International Conference on Program Comprehension (ICPC)*. 2006, pp. 181–190. doi: 10.1109/ICPC.2006.45.
- [16] A. E. Hassan and R. C. Holt. “Predicting Change Propagation in Software Systems”. In: *Proceedings of the 20th IEEE International Conference on Software Maintenance (ICSM)*. 2004, pp. 284–293. doi: 10.1109/ICSM.2004.1357812.
- [17] D. Helm, S. Keidel, A. Kampkötter, J. Düsing, T. Roth, B. Hermann and M. Mezini. “Total Recall? How Good Are Static Call Graphs Really?”. In: *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2024. doi: 10.1145/3650212.3652114.
- [18] Y. Labiche, B. Kolbah and H. Mehrfard. “Combining Static and Dynamic Analyses to Reverse-Engineer Scenario Diagrams”. In: *Proceedings of the 29th IEEE International Conference on Software Maintenance (ICSM)*. 2013, pp. 130–139. doi: 10.1109/ICSM.2013.24.
- [19] LLVM Project. *Clang: a C language family frontend for LLVM*. [Online; accessed June 2026]. 2024. url: <https://clang.llvm.org/>.
- [20] D. Lo, G. Ramalingam, V.-P. Ranganath and K. Vaswani. “Mining Quantified Temporal Rules: Formalism, Algorithms, and Evaluation”. In: *Science of Computer Programming* 77.6 (2012), pp. 743–759. doi: 10.1016/j.scico.2010.10.003.
- [21] C. D. Manning, P. Raghavan and H. Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008. url: <https://nlp.stanford.edu/IR-book/pdf/irbookonlinereading.pdf>.

- [22] Neo4j, Inc. *The Neo4j Cypher Manual*. [Online; accessed June 2026]. 2024. url: <https://neo4j.com/docs/cypher-manual/>.
- [23] Object Management Group. *Systems Modeling Language (SysML) Version 2.0 Specification*. Tech. rep. [Online; accessed June 2026]. Object Management Group, 2024. url: <https://www.omg.org/spec/SysML/2.0/>.
- [24] A. Santos, A. Cunha and N. Macedo. “Static-Time Extraction and Analysis of the ROS Computation Graph”. In: *2019 Third IEEE International Conference on Robotic Computing (IRC)*. 2019, pp. 62–69. doi: 10.1109/IRC.2019.00018.
- [25] W. Shang, Z. M. Jiang, B. Adams, A. E. Hassan, M. W. Godfrey, M. Nasser and P. Flora. “An Exploratory Study of the Evolution of Communicated Information about the Execution of Large Software Systems”. In: *Proceedings of the 18th Working Conference on Reverse Engineering (WCRE)*. 2011, pp. 335–344. doi: 10.1109/WCRE.2011.48.
- [26] T. Systä. *Static and Dynamic Reverse Engineering Techniques for Java Software Systems*. Tech. rep. University of Tampere, Department of Computer Science, 2000. url: <https://api.semanticscholar.org/CorpusID:35172970>.
- [27] K. Taniguchi, T. Ishio, T. Kamiya, S. Kusumoto and K. Inoue. “Extracting Sequence Diagram from Execution Trace of Java Program”. In: *Eighth International Workshop on Principles of Software Evolution (IWPSE’05)*. 2005, pp. 148–154. doi: 10.1109/IWPSE.2005.19.
- [28] M. Theile, O. D. Dantsker, R. Nai, M. Caccamo and S. Yu. “uavAP: A Modular Auto-pilot Framework for UAVs”. In: *AIAA AVIATION 2020 FORUM*. CPSCore framework available at <https://github.com/theilem/cpsCore>. 2020, p. 3268. doi: 10.2514/6.2020-3268.
- [29] N. Walkinshaw, K. Bogdanov, M. Holcombe and S. Salahuddin. “Reverse Engineering State Machines by Interactive Grammar Inference”. In: *14th Working Conference on Reverse Engineering (WCRE 2007)*. 2007, pp. 209–218. doi: 10.1109/WCRE.2007.45.
- [30] M. Weiser. “Program Slicing”. In: *IEEE Transactions on Software Engineering* 10.4 (1984), pp. 352–357. doi: 10.1109/TSE.1984.5010248.
- [31] T. Ziadi, M. A. Almeida da Silva, L. M. Hillah and M. Ziane. “A Fully Dynamic Approach to the Reverse Engineering of UML Sequence Diagrams”. In: *16th IEEE International Conference on Engineering of Complex Computer Systems (ICECCS)*. 2011, pp. 107–116. doi: 10.1109/ICECCS.2011.18.

Acronyms

API Application Programming Interface

AST Abstract Syntax Tree

CDG Control Dependence Graph

DDG Data Dependence Graph

ESI Embedded Systems Innovation

GDB Graph Database

GDI Graph Development Intelligence

IDC Inter-Device Communication

IPC Inter-Process Communication

KerML Kernel Modeling Language

LLM Large Language Model

OMG Object Management Group

PDG Program Dependence Graph

PG Property Graph

ROS Robot Operating System

SysML Systems Modeling Language

TNO Netherlands Organisation for Applied Scientific Research

UML Unified Modeling Language